

How the uDST Works

A complete technical walkthrough of a direct-sampling HF QRP transceiver

Radio designed by Ian Mitchell, VK7IAN

Compiled from the uDST source repository — firmware V2.6.250, PCB Rev 1.1, schematic
2026-09-06

9 September 2026

Table of Contents

1 Read this first.....	6
1.1 What the uDST is	6
1.2 The three processors	6
1.3 How to read this document.....	6
2 The core idea: direct sampling.....	8
2.1 What “direct sampling” means	8
2.2 Why this is a good idea	8
2.3 What it costs	9
3 System map	10
3.1 The one number that sets everything: 31 250 Hz.....	11
3.2 Where the work happens	11
4.1 Antenna to converter: the analogue front end	12
4.1.1 The directional coupler (always in circuit)	12
4.1.2 T/R switching	12
4.1.3 The switched 12 dB attenuator	12
4.1.4 Clamp and switch bias	12
4.1.5 The preselector: four switched high-pass filters	13
4.1.6 The amplifier	13
4.1.7 The converter interface	14
4.2 The ADC	14
4.3 Inside the FPGA: the digital down-converter	14
4.3.1 The NCO: tuning by multiplication	15
4.3.2 The CIC: decimating by 512 with no multipliers	16
4.3.3 The FIR: 255 taps, decimate by 5, unity gain	17
4.4 The gain, noise and headroom budget.....	18
4.4.1 Processing gain: why 12 bits is enough.....	18
4.4.2 Why the shift is exactly 22	19
4.4.3 Sensitivity: why there is 26 dB of analogue gain	19
4.4.4 Headroom: the exact price	20
4.5 The FPGA-to-MCU handoff.....	20
4.5.1 The heartbeat.....	20
4.5.2 The I/Q bus	20
4.5.3 The master side: PIO, not hardware SPI	21
4.5.4 The control surface: one 32-bit number	22
4.6 Inside the MCU: the receive DSP chain.....	22

4.6.1 The frame	22
4.6.2 The SSB chain, stage by stage	23
4.6.3 How the firmware FIRs are built	24
4.6.4 The other demodulators	25
4.6.5 Audio output: PWM, no codec	25
5.1 Microphone to I/Q	26
5.1.1 The microphone processor	27
5.2 CESSB	27
5.3 Inside the FPGA: the digital up-converter	28
5.3.1 Zero-stuffing and the interpolating FIR	29
5.3.2 The interpolating CIC	29
5.3.3 The mixer and the DAC	29
5.4 DAC to antenna: the analogue power stage	30
5.4.1 The DAC	30
5.4.2 Reconstruction filter	30
5.4.3 Driver	30
5.4.4 Power amplifier	30
5.4.5 Transmit gating: one enable line does everything	31
5.4.6 Output low-pass filter bank: four bands, eight relays	31
5.5 SWR and power metering	32
6 Firmware architecture	34
6.1 Two cores, no operating system	34
6.2 The 32 μ s frame	34
6.3 Boot-time self-checks	34
6.4 Cross-core communication	35
6.5 Timing and memory discipline	36
6.5.1 Measured CPU load, on screen	36
6.5.2 Flash avoidance	36
6.5.3 Memory budget	37
6.6 Settings storage	37
7.1 Spectrum display and waterfall	38
7.1.1 Why there are two FFTs in this radio	38
7.1.2 The panadapter chain	38
7.2 AGC and the S-meter	39
7.3 Noise blanker	40
7.4 Noise reduction ("JNR")	41
7.5 Notch filter	42

7.6 CW transmit	43
7.6.1 Envelope shaping	43
7.6.2 Sidetone	43
7.6.3 Keyer	44
7.7 CW decode: two decoders	45
7.7.1 Decoder 1: adaptive single threshold	45
7.7.2 Decoder 2: Schmitt trigger plus asymmetric estimator	46
7.7.3 Morse lookup	46
7.8 FT8	47
7.8.1 Provenance	47
7.8.2 Protocol constants	47
7.8.3 Audio front end	48
7.8.4 Building the waterfall	48
7.8.5 The 2500-point FFT	49
7.8.6 Sync search: finding the signal	49
7.8.7 Soft demapping and LDPC	50
7.8.8 Acceptance and duplicate suppression	50
7.8.9 What is deliberately absent	51
7.8.10 SNR estimate	51
7.8.11 Message packing	51
7.8.12 Encoding and transmission	51
7.8.13 Slot timing without a clock	52
7.8.14 The user interface	53
7.9 Display and user interface	54
8 Hardware reference	56
8.1 The FPGA	56
8.2 The MCU	57
8.3 Power	57
8.4 Support ICs	58
8.5 PCB	59
8.6 Stated performance	60
9 Appendix A: the numbers in one place	62
9.1 Rates and clocks	62
9.2 Bit widths through the DDC	62
9.3 Bit widths through the DUC	63
9.4 Gain accounting, receive	63
9.5 Firmware filter inventory	63

10 Appendix B: pin and port maps	65
10.1 Pico GPIO	65
10.2 FPGA pin assignments	65
10.3 TCA9534 port assignments	66
10.4 Bands	66
11 Appendix C: discrepancies and gaps found in the source	67
11.1 Comment/code mismatches (since fixed)	67
11.2 Behavioural surprises	68
11.3 Documentation gaps	68
11.4 BOM inconsistencies	69
11.5 Architectural notes for a reviewer	69
12 Appendix D: where to look in the source	71
12.1 Repository layout	71
12.2 Verilog	71
12.3 Firmware, by size	71
12.4 Build configuration	72
12.5 Program structure diagram	73
13 Closing note	74

1 Read this first

1.1 What the uDST is

The uDST is a 5 W (typically 6–10 W) HF transceiver covering 80 m through 10 m, plus general-coverage shortwave receive from 0.5 to 30 MHz. It does SSB, CW, AM receive, FT8, and other digital modes. It has a 30 kHz panoramic spectrum display with waterfall, a noise blanker, noise reduction, a tunable notch, CESSB on transmit, a built-in FT8 decoder with QSO sequencer, and two selectable CW decoders.

None of that is unusual for a modern QRP rig. What *is* unusual is how little of it is analogue.

The entire receive analogue chain, antenna to converter, is: a directional coupler, a relay, an optional 12 dB pad, one of four switched high-pass filters, one monolithic amplifier, and a transformer. That is it. There is no mixer, no local oscillator, no IF strip, no crystal filter, no quadrature detector, no audio codec. The antenna signal is digitised whole — the entire 0–40 MHz band at once — and *everything* after that point is arithmetic.

1.2 The three processors

Understanding the uDST means understanding a division of labour between three very different engines.

The analogue front end does one job: present the 0–30 MHz band to a 12-bit converter at a sensible level. It contributes 26 dB of gain, protects the converter from out-of-band energy, and gets out of the way.

The FPGA (a Gowin GW1NR-9 on a small module) runs the two things that must happen at 80 million samples per second: it tunes, and it decimates. On receive it multiplies the incoming stream by a numerically generated sine and cosine — that is the “mixer” — and then throws away 2559 out of every 2560 samples through a filter chain, delivering a 31.25 kHz-wide complex baseband slice as 16-bit I and Q. On transmit it does the exact reverse. The FPGA holds no state you can configure except a single 32-bit number: the tuning word.

The microcontroller (a Raspberry Pi Pico 2 — dual-core RP2350 at 250 MHz) is the radio. It demodulates, filters, blanks noise, notches heterodynes, runs AGC, decodes CW and FT8, draws the spectrum, and drives the display. It runs bare metal: no operating system, no interrupts, no DMA. One core does hard-real-time DSP on a 32-microsecond budget; the other does everything a human can perceive.

Those three engines are joined by two thin wires: a 32-bit SPI frame carrying one I/Q sample each way, and a single square wave from the FPGA telling the MCU when to run.

1.3 How to read this document

Sections 2 and 3 explain the architecture and why it was chosen. Sections 4 and 5 walk the receive and transmit paths end to end — hardware, FPGA, and firmware, in signal order. Section 6 covers firmware architecture. Section 7 takes each user-facing feature and shows exactly how it is implemented. Section 8 is a hardware reference. The appendices hold the number tables, pin maps, and a list of discrepancies found between the source comments and the code.

Sourcing conventions. Every figure quoted here is traced to a specific file in the repository. Where a number is *derived* from component values or code rather than stated in the repository, it says so. Where the source contradicts itself, Appendix C says which one the silicon actually obeys. A small number of statements are standard datasheet behaviour for a named part rather than anything the repository asserts — the AD9236’s output format, the AD9740’s internal reference, the ST7789 panel’s active area, and similar; these are flagged where they matter and listed in Appendix C. Descriptions of the PCB layout are read off the Gerbers and the board renders, since nothing in the repository describes the layout in words.

2 The core idea: direct sampling

2.1 What “direct sampling” means

A conventional receiver reduces the frequency of the signal before it filters or detects it. A superhet mixes the antenna signal down to a fixed intermediate frequency where a good filter can be built. A phasing (Tayloe) receiver mixes it straight down to audio in two channels 90 degrees apart. Both need an analogue local oscillator that tunes, and both need analogue circuitry after the mixer.

A direct-sampling receiver skips the mixer. It puts an analogue-to-digital converter on the antenna and samples the whole band. The uDST samples at 80 MSPS, so by Nyquist it can represent everything from DC to 40 MHz — which comfortably contains all of HF. From that point on the signal exists only as numbers, and tuning becomes multiplication.

The transmit side is the mirror image: a 10-bit DAC clocked at 80 MSPS synthesises the RF directly. There is no modulator, no balanced mixer, no carrier null to adjust.

2.2 Why this is a good idea

The repository makes an extended argument for the architecture. It is worth restating with the reasons made explicit, because each advantage traces to a specific piece of hardware that *isn't there*.

The signal path collapses. Every analogue stage you remove is a stage that cannot drift, cannot mismatch, cannot generate intermodulation products, and cannot pick up mains hum. The uDST's receive chain has one active device in it.

Quadrature is exact by construction. This is the largest single win over the phasing architecture. In a Tayloe rig, opposite-sideband rejection is set by how closely two analogue channels match in gain and phase across the whole audio passband. Component tolerances typically give 40–50 dB, more only with calibration, and it moves with temperature. In the uDST the “mixer” is a numerically controlled oscillator producing sine and cosine that are exactly 90 degrees apart to arithmetic precision, and the I and Q streams then pass through *literally the same* CIC and FIR logic. Image rejection is effectively unlimited. There is nothing to calibrate and nothing to drift.

The zero-IF pathologies vanish. A Tayloe detector lands the wanted signal at DC, where it collides with op-amp offset voltages, 1/f flicker noise, LO leakage, mains hum, and second-order intermodulation products — all of which sit squarely in the passband. The uDST's DDC also produces a zero-IF result, but the mixing happens numerically at 80 MSPS. There is no analogue circuitry at baseband to inject any of it.

Spurious responses are few and fixed. A switching detector responds at odd harmonics of its LO — a strong signal at three times the tuned frequency gets in only about 9.5 dB down, which is why phasing rigs need real preselection. A superhet adds image responses, half-IF spurs, IF breakthrough, and birdies wherever LO harmonics beat together. A direct-sampling receiver's spurious responses are aliases of one fixed Nyquist boundary, handled once by one fixed filter, plus converter spurs that are characterised on the datasheet. Nothing moves as you tune.

One clean fixed clock instead of an agile LO. Close-in dynamic range in a superhet is often limited by reciprocal mixing — the synthesiser’s phase noise smearing a strong adjacent signal across your passband — and any wide-tuning PLL carries spurs. The direct-sampling equivalent of phase noise is sample-clock jitter, and a single fixed-frequency crystal oscillator is about the easiest low-phase-noise source there is. The uDST uses a 24 MHz TCXO multiplied to 80 MHz by one PLL. Tuning happens in the NCO, with 0.0186 Hz resolution, instant settling, and no phase-noise penalty for agility.

Filtering is done properly, once, in numbers. The DDC’s 255-tap FIR gives near-brick-wall selectivity with exactly linear phase. No crystal-filter passband ripple, no blow-by, no group-delay distortion at the band edges, no aging, no cost. Bandwidth is a coefficient set, not a component. Note that even a “DSP” superhet cannot escape this: its DSP only ever sees what survived the analogue roofing filter, so close-in strong-signal performance is still hostage to that one expensive crystal filter.

Processing gain answers the “only 12 bits” objection. See section 4.4 — decimating from 40 MHz of Nyquist bandwidth down to a 31.25 kHz channel buys about 31 dB, roughly five extra effective bits, so the 12-bit converter behaves like a 17-bit one in-channel.

The whole band is available at once. A full-span waterfall, several simultaneous receive slices, wideband noise blanking, diversity reception — all of it becomes possible once the band is digitised. A phasing rig’s panadapter is capped at its codec’s sample rate; a superhet sees one roofing-filter slice at a time.

Transmit is symmetric. The DUC synthesises SSB or FT8 directly at RF with mathematically perfect carrier and sideband suppression. No balanced-modulator nulling, no transmit filter alignment.

And it repeats. Performance lives in the Verilog, so unit two behaves exactly like unit one. There are no trimmers in the signal path and no alignment procedure. The only two adjustments on the whole board are the PA gate-bias trimmers.

2.3 What it costs

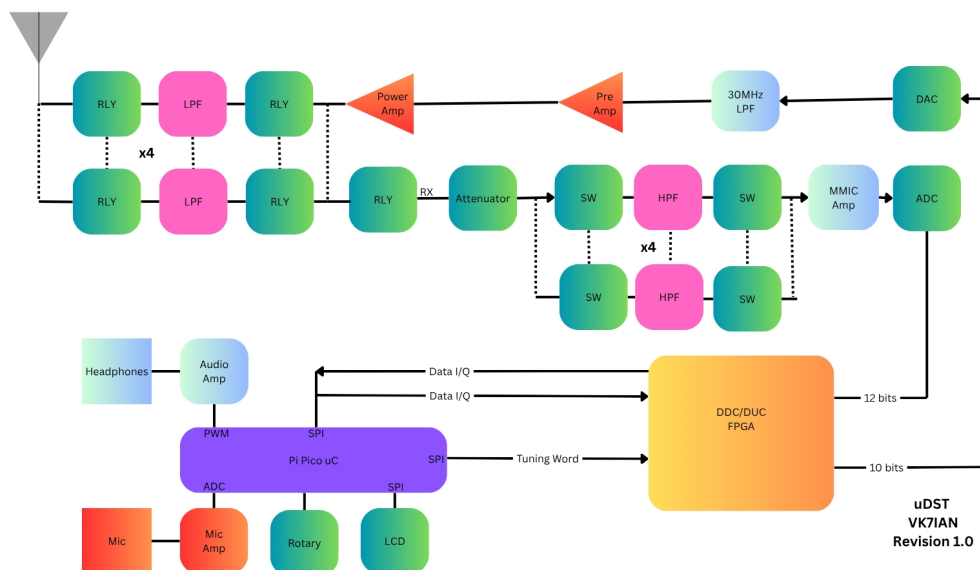
Direct sampling is not free, and several of the uDST’s design choices are direct responses to the two prices you pay.

No channel filter ahead of the converter. The ADC’s clipping point has to absorb the *aggregate* power of everything from 0.5 to 30 MHz simultaneously. On a large antenna at night, the sum of the international broadcast bands can run to –30 or even –20 dBm. This is why the uDST has a switched 12 dB attenuator, four switched high-pass filters, and a selectable digital scaling shift.

Gain trades sensitivity against headroom, exactly. Every dB of gain ahead of the converter improves the noise figure by a dB and lowers the clipping point by a dB. There is no way around it, only a place to sit. Section 4.4 shows where the uDST chose to sit, and why that spot is not a coincidence.

3 System map

Figure 1 is the author's own top-level block diagram.



The uDST block diagram, Revision 1.0 ([docs/uDST.png](#)). Top row is transmit, right to left; second row is receive, left to right. Each filter bank is drawn once with an “x4” brace to indicate its four switched sections.

Read it as follows.

Antenna node. The BNC connector feeds a node shared by three things: the in-line directional coupler (always present, in both directions), the output of the transmit low-pass filter bank, and the receive T/R relay. Note carefully that the receive tap is on the *antenna* side of the low-pass bank — the receiver does not pass through the transmit filters. The deselected filter sections sit as grounded stubs on the node whether receiving or transmitting on one of the other bands.

Receive, left to right. T/R relay, then the switched 12 dB attenuator (two relays and a pi network), then a 4:1 FET multiplexer selecting one of four high-pass filters, then a second multiplexer collecting the output, then the MMIC amplifier, then a 1:4 impedance transformer feeding the 12-bit ADC differentially. The parallel 12-bit bus goes straight into the FPGA module.

Transmit, right to left along the top. The FPGA drives a 10-bit DAC, whose two complementary current outputs each pass through a 7-element reconstruction low-pass, then into the two halves of a dual wideband op-amp acting as the driver, then into a push-pull pair of RD16HHF1 MOSFETs, then through the output transformer into one of four relay-switched low-pass filters, and out to the antenna node.

The control cluster. The FPGA talks to the Pico over two independent SPI buses: a bidirectional one carrying I/Q sample data, and a write-only one carrying the tuning word.

The Pico additionally drives the display over SPI, reads the encoder, generates receive audio as PWM into an audio power amplifier, and reads the microphone through its own on-chip ADC. Band filter selection and the attenuator go out over I2C to a GPIO expander driving a Darlington array driving the relay coils.

3.1 The one number that sets everything: 31 250 Hz

The single most important constant in the whole design is the DDC output rate.

$$\frac{80 \text{ MHz}}{512 \times 5} = \frac{80\,000\,000}{2560} = 31\,250 \text{ Hz}$$

That rate is the sample rate of *everything* downstream. It is the I/Q rate on the SPI bus, the audio sample rate, the rate at which the microphone is sampled, the rate the receive audio PWM is updated, the rate the encoder is polled, and the rate every firmware filter is designed for. It appears in the firmware exactly once, at `src/util.h:4`:

```
#define SAMPLERATE 31250u
```

At 250 MHz, one sample period is 8000 CPU clock cycles, or 32 microseconds. That is the hard deadline the DSP core lives under, and it explains most of the firmware's stylistic choices.

3.2 Where the work happens

Function	Where	Why there
Tuning (frequency translation)	FPGA NCO	Must run at 80 MSPS
Decimation, 2560:1	FPGA CIC + FIR	Must run at 80 MSPS
Interpolation and RF synthesis	FPGA DUC	Must run at 80 MSPS
Sideband selection (Hilbert)	MCU core 1	Cheap at 31.25 kHz, and wants floats
Channel filtering, AGC, NB, NR, notch	MCU core 1	32 µs budget is ample
CW keying and envelope shaping	MCU core 1	Sample-accurate but slow
Spectrum FFT	MCU core 0	1024-point, 30 times a second
FT8 decode (FFT, sync search, LDPC)	MCU core 0	2.36 s of the 15 s slot
Display, menu, metering	MCU core 0	20 frames per second
Band filter and attenuator selection	MCU core 0 via I2C	Only changes when you tune

The split is not arbitrary. Anything that must see every 80 MHz sample is in the FPGA. Anything that must happen on every 31.25 kHz sample without exception is on core 1. Everything a human notices is on core 0, where a missed deadline costs a dropped display frame rather than a click in the audio. # 4 The receive path, end to end

4.1 Antenna to converter: the analogue front end

4.1.1 The directional coupler (always in circuit)

Immediately after the BNC (J1) sit two identical ferrite transformers, T1 and T2, both FT37-43 cores wound 1 turn to 10 turns. This is a tandem-match style directional coupler and it stays in circuit in both directions. Its two output legs are detected by 1N5711W Schottky diodes (D2, D3) into 49.9 ohm terminations (R11, R12), then RC-filtered into two separate 10-bit I2C ADCs:

- **Forward** leg: D2, R11, then R6/C21/C22/R9/R10 into U1, an MCP3021A0T at address 0x48.
- **Reflected** leg: D3, R12, then R7/C23/C24/R13/R14 into U2, an MCP3021A1T at address 0x49.

The firmware only reads these while transmitting. Section 5.6 covers the SWR and power maths.

4.1.2 T/R switching

T/R is done with a single reed relay, **K9**, a **Littelfuse HE721C1210** (the schematic notes COMUS 3563.1231 as an alternative) — a hermetically sealed 1 Form C part with an internal electrostatic shield. The coil is driven low-side from the **TXENn** net, so it is energised on transmit. Only “pin 14 to ground” is clearly legible in the schematic; the rest of the pin assignment (pin 1 to the antenna node, pins 7 and 8 tied together going on to the attenuator) is read from a zoomed render and is worth confirming against the board. Functionally the arrangement is unambiguous: on receive the front end is connected to the antenna, on transmit the front end is **grounded**.

There are **no PIN diodes anywhere in the design**, and there is no separate antenna-changeover relay — the PA output transformer and the low-pass bank stay connected to the antenna node at all times. Receiver protection is K9 alone.

4.1.3 The switched 12 dB attenuator

Two more HE721C1210 reed relays (K10, K11), both coils driven from the same **ATTEN** net, select either a straight bypass or a resistive pi network: R25 100 ohm shunt, R27 100 ohm series, R28 100 ohm shunt. That network gives **−12.04 dB** with an input impedance of about 57 ohms into a 50 ohm load (derived from the component values; the repository just calls it “a 12 dB attenuator”).

On a large antenna this pad is the primary defence against the aggregate-power problem described in section 2.3.

4.1.4 Clamp and switch bias

Between the attenuator and the preselector: series 10 nF blocking caps (C52, C53), a pair of 1N4148W diodes to ground (D4, D5) as an overload clamp, and a 2.5 V bias node (R24/R26 from the 5 V rail, bypassed by C61) injected through a 10 μ H choke, L20. The bias exists to put the CMOS FET switches of the preselector multiplexers into their linear region.

4.1.5 The preselector: four switched high-pass filters

This is the part most likely to surprise someone expecting a conventional radio. The uDST's preselector is not a band-pass bank — it is four **high-pass** filters, each a 5th-order LC ladder (three series capacitors, two shunt inductors). The low-pass filters stay connected and are in series with the high-pass filters when receiving, thus forming a band-pass filter.

The reasoning is direct-sampling-specific. A high-pass filter's job here is to keep the strong low-band energy — AM broadcast, and everything below the band you are actually using — out of the converter's headroom budget. Since the converter can see the whole band regardless, the only thing preselection has to accomplish is *headroom management*, and the biggest offenders are always below you.

Switching is by two **SN74CBT3253** dual 4:1 FET multiplexers: U6 selects the input, U5 collects the output, both halves of each device wired in parallel, both output-enables tied permanently low. The select lines S0 and S1 come directly from the I2C GPIO expander as `HPF1` and `HPF2`.

Section	Mux channel	Series C1	Shunt L1	Series C2	Shunt L2	Series C3
3 MHz	B4	C44 1 nF	L13 2.2 μ H	C45 510 pF	L14 2.2 μ H	C46 1 nF
4 MHz	B3	C54 820 pF	L15 1.2 μ H	C55 470 pF	L16 1.2 μ H	C56 820 pF
8 MHz	B2	C62 270 pF	L21 680 nH	C63 180 pF	L22 680 nH	C64 270 pF
16 MHz	B1	C68 120 pF	L27 330 nH	C69 82 pF	L28 330 nH	C70 120 pF

Selection is by tuned frequency, in `set_filter()` at `src/uDST.ino:581`:

Tuned frequency	Section	HPF1 (S0)	HPF2 (S1)
below 5 MHz	3 MHz	high	high
5 to 8 MHz	4 MHz	low	high
8 to 16 MHz	8 MHz	high	low
16 MHz and above	16 MHz	low	low

`set_filter()` keeps static copies of the last-selected code so it only generates I2C traffic on an actual transition. Note the consequence of the lowest section: below about 3 MHz the receiver is rolling off, which is exactly what the README means by “reduced sensitivity below 3 MHz”.

4.1.6 The amplifier

One device: **U7, a Qorvo AG203-63G MMIC** in SOT-363. Pin 3 in, pin 6 out, all other pins grounded. It is biased in the classic MMIC fashion — 5 V through R29 (49.9 ohm) into a decoupled node, then through a 10 μ H choke L26 into the output pin, with C75 blocking DC on the way out. With the schematic's annotated device voltage of about 2.8 V, that resistor sets roughly **44 mA** of bias current.

The repository states its gain as **20 dB** (`docs/README.md`). The MMIC's noise figure is 3.1 dB; the system noise figure worked out from the whole chain (section 4.4) builds on that figure.

4.1.7 The converter interface

T5 is a trifilar transformer, part 78602/4JC-2 (Mini-Circuits T-622-KK81+ is a tested alternative per `BOM/README.md`). It is a **1:2 turns, 1:4 impedance** transformer, so it contributes **6 dB of voltage gain** — explicitly described as such in `docs/README.md`.

Its wiring around the ADC is worth understanding because it does four jobs at once:

- Primary terminated by R32 (49.9 ohm) to ground, presenting a 50 ohm load to the MMIC.
- Secondary drives `VINP` and `VINN` through R31 and R33 (49.9 ohm each), damping the transformer and isolating the converter's switched-capacitor input.
- C78, 22 pF differentially across the inputs.
- Common-mode bias: R30 (1 kohm) from 3.3 V to one secondary end, R34 (1 kohm) from the other end to ground, bypassed by C79. Because the winding is a DC short, **both ADC inputs sit at 1.65 V** = $AVDD/2$, which is exactly what the AD9236 wants.

4.2 The ADC

U9 is an Analog Devices AD9236BRURL7-80: 12 bits, 80 MSPS maximum, clocked here at exactly 80 MHz.

The interface is **parallel single-ended CMOS**, D0 through D11 on pins 15–28, wired straight into the FPGA module. Not LVDS, not serial. `DRVDD` is on the 3.3 V rail so the levels are 3.3 V LVCMOS, which is what the constraints file declares for every `adc_in` pin.

Two strap pins set its behaviour:

- **SENSE tied to AGND** selects the internal 1.0 V reference, giving a **2 V peak-to-peak differential full-scale span**. (The connection is read from the schematic; the 2 V p-p consequence is standard AD9235/AD9236 family behaviour.)
- **MODE** selects the output format, and the format in use is **two's complement** — which is how the Verilog declares the port: `input wire signed [11:0] adc_in, // AD9236, two's complement`. (The MODE pin's net is not legible in the schematic; the strap-to-format mapping is datasheet behaviour.)

`PDWN` is grounded, `OTR` is unused, and the reference pins carry the full ADI-recommended decoupling (C80/C81 to ground on `REFB/REFT`, plus C76/C77 differentially between them).

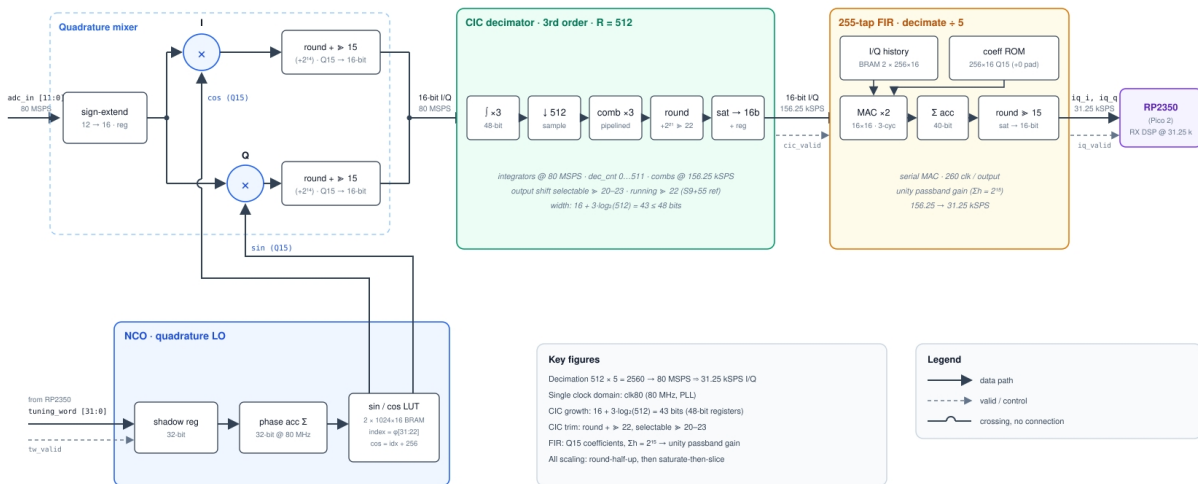
There is **no amplifier driving the converter**. The MMIC drives the transformer, the transformer drives the converter through two resistors. That is the whole thing.

4.3 Inside the FPGA: the digital down-converter

The DDC lives in `verilog/DDC26.v`. Its own header states the chain and the gain accounting; this section unpacks both.

uDST — FPGA Digital Down-Converter (RX)

DDC25.v · ddc_top · single 80 MHz clock domain (clk80, PLL) · 12-bit ADC @ 80 MSPS in → 16-bit I/Q @ 31.25 kSPS out (decimation 512 × 5 = 2560)



The DDC block diagram from docs/uDST_DDC25_block_diagram.png.

ADC (80 MSPS, 12 bit signed)
 → NCO quadrature mixer → I, Q at 80 MSPS
 → 3-stage CIC, decimate by 512 → I, Q at 156.25 kHz
 → 255-tap FIR, decimate by 5 → I, Q at 31.25 kHz, 16 bit signed

4.3.1 The NCO: tuning by multiplication

A numerically controlled oscillator is a counter and a lookup table. Add a constant (the *tuning word*) to a 32-bit accumulator every clock; the accumulator's value is the phase, and its top bits index a sine table. In `nco_sincos_rx`:

```
reg [31:0] phase;
always @(posedge clk) phase <= phase + tuning_word;

wire [9:0] sin_idx = phase[31 -: 10]; // top 10 bits -> 1024-entry table
wire [9:0] cos_idx = sin_idx + 256; // +90 degrees, wraps mod 1024
```

The output frequency is $f_{out} = f_{clk} \times TW/2^{32}$, so the tuning resolution is

$$\frac{80 \text{ MHz}}{2^{32}} = 0.018626 \text{ Hz}$$

Two things in this module are worth noting as FPGA craft rather than DSP.

First, **there are two copies of the same 1024-entry table**, `sin_lut` and `cos_lut`, both loaded from `sin_lut.hex`. They are identical — the cosine is obtained by adding 256 to the index. The duplication exists purely to get two independent read ports so sine and cosine can be fetched in the same clock cycle.

Second, **the reads are registered**, not combinational, and the comment explains why: a Gowin block RAM read is synchronous, so a registered read infers BSRAM cleanly. An asynchronous read of a 1024x16 ROM does *not* infer block RAM — the synthesiser builds it out of a large, slow multiplexer made of logic cells instead, and timing closure at 80 MHz goes away.

The mixer itself is two multipliers. The 12-bit ADC word is sign-extended to 16 bits and multiplied by cosine (giving I) and sine (giving Q). The multiplies are written inline with input registers and output registers, again so Gowin packs them into hard DSP blocks:

```
adc_r <= {{4{adc_in[11]}}, adc_in}; // sign-extend 12 -> 16
mult_i_raw = $signed(adc_r) * $signed(cos_r);
mult_q_raw = $signed(adc_r) * $signed(sin_r);
mixI <= (mult_i_reg + 32'sd16384) >>> 15; // Q15 -> integer, round half up
mixQ <= (mult_q_reg + 32'sd16384) >>> 15;
```

Because the sine table is Q15 (values scaled by 32768), the product is Q15 and gets shifted back by 15 with a round-half-up constant of 2^{14} . Note also the fundamental gain fact of quadrature mixing a *real* signal: a carrier $A\cos\omega t$ becomes a complex envelope of amplitude $A/2$, so the mixer has an inherent gain of **0.5**. That factor reappears in the gain budget.

4.3.2 The CIC: decimating by 512 with no multipliers

A cascaded integrator-comb filter is the standard way to decimate hard by a large factor using only adders. The trick is elegant: run N cascaded accumulators at the high rate, drop samples, then run N cascaded differencers at the low rate. Mathematically the accumulators and differencers form a moving-average filter of length R raised to the N th power, but you never build the taps.

The uDST uses $N = 3$ stages and $R = 512$. The integrators run at 80 MHz:

```
reg signed [47:0] i1I, i2I, i3I;
always @(posedge clk80) begin
    i1I <= i1I + mixI;
    i2I <= i2I + i1I;
    i3I <= i3I + i2I;
end
```

Decimation is a free-running 9-bit counter; on `dec_cnt == 511` the integrator outputs are sampled and the three comb stages are computed and the output is scaled and registered.

Two details in this code carry real engineering weight, and the source comments call both out explicitly.

Integrators and combs must be the same width. They are 48 bits on both sides. The comment explains why in one sentence that is worth reproducing:

A CIC is only correct because the integrators' modulo- 2^W wraparound is exactly undone by the combs; if the combs were wider, every wrap would be sign-extended into the wider arithmetic instead of cancelling.

This is the classic CIC gotcha. The integrators *will* overflow — they are unbounded accumulators — and that is fine, because two's complement subtraction in the combs undoes the wrap exactly. Widen the combs and you break the identity.

The width is sized from the worst case. Peak mixer output is 2048, and the CIC's DC gain is $R^N = 512^3 = 2^{27}$, so the peak integrator value is about 2.75×10^{11} , needing 39 bits including sign. 48 bits leaves margin.

The output shift sets the gain, and it is selectable. The CIC's 2^{27} gain has to be scaled back down. The active line is:

```
cicI_reg <= (y2I + 48'sd2097152) >>> 22; // S9 + 55
```

which keeps $2^{27-22} = 32$, i.e. five bits of the CIC's gain. Three alternative shifts are present as commented-out lines — `>>>23` (“ADC clips first”), `>>>21` (“S9 + 49”) and `>>>20` (“S9 + 43”) — each with its matching round constant of 2^{S-1} . The source is emphatic that the constant must change with the shift. Section 4.4 explains why `>>>22` is the calibrated choice.

The comb stages are pipelined across the same clock in a deliberate staircase (`i3I_s` sampled, then `y0`, `y1`, `y2` each computed from the previous stage's delayed value) so that the result is stable before `cic_valid` asserts. The comment notes a one-sample offset that this introduces — the registered outputs reload one clock after `cic_valid` goes high, so the FIR stores the *previous* decimated sample. That is a constant 1/156250 s delay on the entire stream, and the comment exists so that nobody rediscovers it as a bug.

Finally the 32-bit CIC result is **saturated** to 16 bits, not truncated: `>>>22` can put the FIR input at full scale, so honest clipping is preferable to a silent wrap.

4.3.3 The FIR: 255 taps, decimate by 5, unity gain

The CIC's frequency response is a cascade of sinc-like lobes — fine for gross decimation, but it droops across the passband and its stopband rejection is mediocre. The FIR that follows does three jobs: it corrects the CIC's droop, it provides the real anti-alias protection for the final 5:1 decimation, and it defines the 31.25 kHz channel.

`fir255_dec5_rx` is a **serial** FIR: one multiplier per channel, working through 255 taps one per clock cycle. It has 512 clock cycles between input samples (156.25 kHz into an 80 MHz clock) so a serial implementation fits comfortably, and it saves 254 multipliers.

The coefficients are Q15 and sum to exactly 32768, so the `>>>15` at the output gives **unity passband gain**. Both sample histories (256-word buffers holding 255 taps' worth of history, 16 bits, two channels) and the coefficient table live in block RAM, because they will not fit in flip-flops on a device this size. The buffer is 256 rather than 255 so that the 8-bit address wraps for free.

The pipeline latency accounting is the part that must be exact, and the source spells it out:

```
present a tap address on MAC clock k
BRAM registered read      -> operands valid at k + READ_LAT      (1)
mac16_reg (3 stages)     -> product valid at k + 1 + MUL_LATENCY (3)
round register           -> + ROUND_LATENCY                     (1)
CNT_LAST = 255 + 1 + 3 + 1 = 260
```

So the accumulator is latched 260 clocks after the run starts. Because the pipeline is always accumulating — unconditionally, on every clock — there must be no garbage in it. The trick is that the coefficient address is **forced to a zero-valued ROM entry** whenever a real tap is not being issued:

```
wire [7:0] coeff_addr = issuing ? cnt[7:0] : ZERO_TAP; // ZERO_TAP = 255
```

Hence the warning in the header: `fir_rx255_coeffs.hex` must contain **256** words — the 255 real taps *plus* a final `0000`. `Hmem[255]` is read on every idle clock, so if that entry is missing the accumulator is poisoned permanently. It shows up as X in simulation and only survives on hardware because Gowin BSRAM initialises to zero.

Two more choices are timing-driven and are documented as such:

- The multiplier `mac16_reg` registers its **inputs** as well as its output, so the multiply runs register-to-register and packs into the DSP block's own pipeline. Registering only the output would leave the full combinational multiply in one clock and Fmax would collapse.
- The round-half-up add sits in its **own register stage**. The comment records that putting the 40-bit add on the same clock as the accumulate cost about 20 MHz of Fmax when it was first tried.

The accumulator is 40 bits (reduced from 48 — the comment notes about 32 bits is the real worst case), and the output is saturate-then-slice, again clipping honestly rather than wrapping.

4.4 The gain, noise and headroom budget

This is the analysis that justifies every scaling choice in the design, and `docs/README.md` works it through in detail. It is worth following because it ends in a coincidence that is not a coincidence.

4.4.1 Processing gain: why 12 bits is enough

An ideal 12-bit converter has an SNR of $6.02 \times 12 + 1.76 = 74.0$ dB. But that noise is spread across the whole 40 MHz Nyquist band, so as a *density* it is

$$-74 - 10\log_{10}(40 \times 10^6) = -150 \text{ dBFS/Hz}$$

The DDC does not remove any noise. It discards the 39.97 MHz of spectrum you did not want, and the noise that was in that spectrum goes with it. What remains in the 31.25 kHz output is

$$-150 + 10\log_{10}(31\,250) = -105 \text{ dBFS}$$

So the processing gain is

$$10\log_{10} \left(\frac{40 \text{ MHz}}{31.25 \text{ kHz}} \right) = 10\log_{10}(1280) \approx 31.1 \text{ dB}$$

split as 24.1 dB from the CIC ($\div 512$, exactly 4.0 bits) and 7.0 dB from the FIR ($\div 5$, 1.2 bits). That is **5.2 bits of genuine extra resolution**: the 12-bit converter behaves like a ~ 17 -bit converter inside the 31.25 kHz channel. Firmware channel filtering down to a 2.4 kHz SSB bandwidth adds a further $10\log_{10}(31250/2400) \approx 11$ dB on top of that.

One caveat the repo raises and then dismisses: processing gain assumes the quantisation noise is decorrelated across the band, which requires the least significant bits to be genuinely exercised. Since band noise arrives some 20 dB above the converter's per-Hz floor, **the converter is continuously dithered by the antenna itself**, and the arithmetic holds without any added dither circuit.

4.4.2 Why the shift is exactly 22

Now the word-growth bookkeeping. The CIC's DC gain is 2^{27} ; the `>>>22` keeps $\times 32$, five bits. Those five bits are precisely the 5.2 bits the decimation earned — so the 16-bit I/Q word is filled with real information rather than padded least significant bits.

Better still, it calibrates the clipping point. Total ADC-code-to-I/Q-code gain is:

Stage	Gain
Quadrature mixing of a real signal	$\times 0.5$
CIC $512^3 = 2^{27}$, output <code>>>>22</code>	$\times 32$
FIR, Q15 coefficients summing to 32768, <code>>>>15</code>	$\times 1$
Net	$\times 16$

A full-scale 12-bit input of ± 2047 therefore lands essentially exactly at 16-bit full scale.

The two source files disagree slightly on the last digit: `docs/README.md` says $\pm 32\,752$ (“16-bit full scale within 0.004 dB”), while the `DDC26.v` header says $\pm 32\,734$ and marks it “all confirmed in simulation”. The difference is about 0.005 dB and comes down to Q15 rounding through the chain; the simulated figure is the one to trust. Either way the digital clip point sits about one LSB below the converter's own. This is not luck; it is the reason for the choice, and the header states it plainly: one more bit of CIC gain (`>>>21`) would clip 6 dB *before* the ADC does.

The 16-bit interface floor is -98 dBFS, about 7 dB above the -105 dBFS theoretical floor, so **the pipe, not the converter, is the final limit** — and 98 dB of dynamic range in 31.25 kHz is far beyond anything HF propagation delivers.

4.4.3 Sensitivity: why there is 26 dB of analogue gain

A bare 12-bit converter clipping near +8 dBm has a noise density of

$$8 - 74 - 10\log_{10}(40 \times 10^6) = -142 \text{ dBm/Hz}$$

Against the -174 dBm/Hz thermal floor that is an effective **noise figure of about 32 dB**. That is hopeless: quiet-rural band noise at 7 MHz is around -144 dBm/Hz, so the bare converter's own quantisation noise would sit *at or above* the band noise, and weak signals on a quiet band would simply be lost in it.

Gain ahead of the converter fixes this by Friis: it divides the converter's noise contribution by the gain. The uDST has **26 dB** — 20 dB from the MMIC plus 6 dB from the transformer. Antenna-referred, the converter's floor drops from -142 to -168 dBm/Hz, its NF contribution falls to about 6 dB, and the system noise figure becomes essentially the amplifier's own NF plus a few dB — call it **8 to 10 dB overall**. In a 2.4 kHz SSB bandwidth that is an MDS of roughly **-131 to -134 dBm**, versus about -108 dBm with no gain at all.

The important framing, from `docs/README.md`: the gain adds no SNR to any signal already above the amplifier's own noise. Its entire job is to stop the converter from *subtracting* SNR.

4.4.4 Headroom: the exact price

Every dB of gain lowers the antenna-referred clipping point by a dB. With full scale near +8 dBm at the converter, 26 dB of gain puts clipping at **−18 dBm at the antenna** — which is **S9 + 55**, exactly the >>>22 label in the Verilog. The digital scaling and the analogue gain plan agree to within a fraction of a dB.

That also explains why the selectable shift is the right companion knob: each step of >>>20 through >>>23 moves digital full scale in 6 dB steps (S9+43, +49, +55, ...), so a future change in analogue gain or a different attenuator can be re-centred without touching the datapath.

Practically, −18 dBm of clip headroom has to absorb the aggregate power of the whole 0–40 MHz band, since there is no channel filter ahead of the converter. With typical HF antennas the night-time broadcast aggregate runs perhaps −30 to −20 dBm, so the margin is real but not extravagant. That is the classic direct-sampling trade, and it is why an attenuator ahead of a large antenna is worth having.

Summary: 26 dB of analogue gain buys about 23 dB of sensitivity, costs exactly 26 dB of clip headroom (landing full scale at S9+55), leaves every digital number unchanged, and makes the band — not the converter — the noise floor. Which is the whole design goal.

4.5 The FPGA-to-MCU handoff

4.5.1 The heartbeat

The FPGA generates a 50% square wave on `data_ready`, phase-locked to the DDC output tick (verilog/radio_top.v):

```
localparam [11:0] HB_HALF = 12'd1280;
always @(posedge clk80_int) begin
    if (rx_iq_valid)    begin data_ready <= 1'b1; dr_cnt <= 0; end
    else if (data_ready) begin if (dr_cnt == HB_HALF-1) data_ready <= 1'b0;
                             else dr_cnt <= dr_cnt + 1; end
end
```

1280 cycles at 80 MHz is 16 µs, so the line is high for 16 µs and low for 16 µs: a 31.25 kHz square wave whose rising edge *is* the new sample. This one wire is the entire scheduling mechanism for the DSP core (section 6.2).

The firmware verifies the pulse train at boot before doing anything else — `check_fpga_data_ready()` at `src/uDST.ino:539` counts GPIO edges over a 10 ms window, expects 625, accepts 312 to 1250, and calls `error_stop(ERROR_FPGA)` (blink code 4) otherwise.

4.5.2 The I/Q bus

`spi_iq_slave` is a 32-bit SPI slave, mode 0, MSB first, CS active low, carrying **one {I[15:0], Q[15:0]} sample per frame**. It is bidirectional in the sense that it changes direction with the `tx_rx` level rather than doing both at once:

- **Receive** (`mode_tx = 0`): on `rx_valid` the latest DDC sample is latched into `rx_hold`; on CS falling that word is loaded into the shifter and streamed out on MISO.
- **Transmit** (`mode_tx = 1`): MOSI is shifted in; on CS rising the word is presented to the DUC with `tx_valid`.

Both SPI slaves **oversample the SPI clock in the 80 MHz domain**. `sck`, `mosi` and `cs_n` are each two-flop synchronised and edge-detected, which means there is no second clock

domain in the design and no asynchronous-clock timing constraint to satisfy. The trade-off, stated in the header: keep the Pico's SPI clock modest — a few MHz — so that a bit period spans several 80 MHz cycles and MISO is stable before the master samples it. At 31.25 kHz times 32 bits the payload is about 1 Mbit/s, so anything from 2 to 6 MHz is ample. The firmware uses 6 MHz.

4.5.3 The master side: PIO, not hardware SPI

The Pico drives the I/Q bus with a **hand-written PIO program**, not the hardware SPI peripheral (`src/asyncSPI.h`). Ten instructions, side-set driving SCK:

```
0: pull block      side 0      ; wait for a 32-bit word, CS idle high
1: set  pins,0     side 0      ; assert CS low
2: set  x,31       side 0
3: out  pins,1     side 0      ; MOSI bit, SCK low
4: nop            side 1      ; SCK rising edge
5: nop            side 1      ; extra MISO settle time
6: in   pins,1     side 1      ; sample MISO, SCK high
7: jmp  x--,3      side 0      ; SCK falling
8: set  pins,1     side 0      ; CS high
9: jmp  0          side 0
```

Five PIO cycles per SCK period, so the divider is $250 \text{ MHz} / (6 \text{ MHz} \times 5) = 8.333$, giving a 30 MHz PIO clock and a 6.0 MHz SCK. A full 32-bit frame takes about 165 PIO cycles, roughly **5.5 μs** — about 17% of the 32 μs frame.

Two things about this are deliberate. The shift direction is configured **left** with autopull and autopush at 32 bits, because PIO's default right-shift would deliver the word bit-reversed. And the transfer primitive is **non-blocking and pipelined**:

```
inline uint32_t AsyncSPI::doTransfer(const uint32_t tx)
{
    if (!pio_sm_is_tx_fifo_full(pio_, sm_)) { pio_sm_put(pio_, sm_, tx); }
    if (!pio_sm_is_rx_fifo_empty(pio_, sm_)) { return pio_sm_get(pio_, sm_); }
    return 0u;
}
```

The value it returns is the previous frame's result. That is one sample — 32 μs — of latency, and it matches the FPGA's hold register. The DSP core never waits for the bus.

Unpacking on the firmware side:

```
const uint32_t rx_sample = iqSPI.doTransfer(0);
const int16_t ii = (int16_t)(rx_sample >> 16);
const int16_t qq = (int16_t)(rx_sample & 0xffff);
```

matching the FPGA's `rx_hold <= {rx_i, rx_q};` — **I in bits 31:16, Q in bits 15:0.**

Transmit is deliberately crossed over, and this is easy to miss. The firmware packs its own `tx_q` into the *high* half:

```
tx_word = ((uint32_t)(uint16_t)tx_q << 16) | (uint16_t)tx_i;
```

while the FPGA unpacks the high half as `tx_i (tx_i <= sr[31:16]; tx_q <= sr[15:0]);`. So firmware-`tx_q` becomes FPGA-`inI` and vice versa. Combined with the per-mode argument swap in `process_mic()`, this crossover is part of how transmit sideband is selected against the DUC's $I\cos\omega t - Q\sin\omega t$ sense. It is not a bug, but anyone tracing I and Q through the transmit path needs to know the halves swap meaning at the bus.

4.5.4 The control surface: one 32-bit number

The tuning word travels on a completely separate bus, `spi_tw_slave`: 32-bit, mode 0, MSB first, **write-only**, latched on the CS rising edge. On the Pico side it uses hardware SPI1 at 8 MHz with a bit-banged chip select, because the FPGA slave has no MISO to read back (`src/tuningWordSPI.h`). A write is a blocking four-byte transaction, roughly 4 μ s.

There are no other control registers. The single 32-bit tuning word is the *entire* configurable state of the FPGA, plus the `tx_rx` level on one GPIO. Bandwidth, mode, sideband, AGC, filters, metering — none of it exists in the FPGA. Sideband selection, for instance, is done by *swapping which firmware variable goes into which half of the word*, not by any FPGA setting.

Frequency computation, `set_frequency()` at `src/uDST.ino:804`, works in **centi-hertz** to keep integer precision:

```
constexpr uint64_t TW032 = 1ULL << 32;
constexpr uint64_t DENOM = FPGA_CLK * 100ULL; // 8e9
const uint64_t f_centihz = 100ull * (radio.frequency + rx_cwoffsetw) + ft8_offset;
const uint32_t tuning_word = (uint32_t)((f_centihz * TW032 + DENOM/2) / DENOM); // round
to nearest
tw.write(tuning_word);
```

The same dial drives both the DDC and the DUC — `tuning_word` fans out to both cores in `radio_top.v` — which is why transmit and receive can never be on different frequencies by accident. CW offset and the FT8 tone offset are folded into this one computation.

Note the consequence for FT8 transmit (section 7.8): each of the 79 symbols is sent by writing a fresh tuning word, once every 160 ms.

4.6 Inside the MCU: the receive DSP chain

Everything from here runs on **core 1**, once per 32 μ s frame, in `loop1()`.

4.6.1 The frame

```
if (digitalRead(PIN_DR) == LOW) { return; } // wait for the FPGA's rising edge
const uint32_t cpu_start = time_us_32();
... read mic ADC ...
... write the audio PWM for the PREVIOUS sample ...
... exchange one 32-bit SPI frame ...
... demodulate ...
const uint32_t cpu_end = time_us_32();
cpu.usage += (cpu_end - cpu_start); cpu.count++;
while (digitalRead(PIN_DR) == HIGH) { tight_loop_contents(); } // wait for the falling
edge
```

Note the ordering: the audio PWM level is written at the **top** of the frame, before any DSP work, so its update is jitter-free regardless of how long the processing takes.

Mode dispatch (`src/uDST.ino:2447`) is where sideband selection actually happens — look at the argument order:

```

case MODE_LSB: dac_audio = DSP::process_ssb(qq,ii,jnr,bw,nb); break;
case MODE_USB: dac_audio = DSP::process_ssb(ii,qq,jnr,bw,nb); break;
case MODE_AM:  dac_audio = DSP::process_am (ii,qq,jnr);      break;
case MODE_CWL: dac_audio = DSP::process_cw (qq,ii,cwsig);    break;
case MODE_CWU: dac_audio = DSP::process_cw (ii,qq,cwsig);    break;
case MODE_DGL: dac_audio = DSP::process_dig(qq,ii);          break;
case MODE_DGU: dac_audio = DSP::process_dig(ii,qq);          break;
case MODE_FT8: ft8_audio = DSP::process_ft8(ii,qq);          break;

```

Swapping I and Q conjugates the complex signal, which flips which sideband survives the Hilbert subtraction. That is the whole of the LSB/USB mechanism: an argument order.

4.6.2 The SSB chain, stage by stage

DSP::process_ssb() in src/dsp.h:212:

```

float ii = FILTER::dc1f((float)in_i / 32768.0f); // 1. scale to +/-1.0, block DC
float qq = FILTER::dc2f((float)in_q / 32768.0f);
noise_blanker(ii, qq, nb_level);                // 2. impulse removal, raw I/Q
const float p45 = FILTER::fap1f(ii);            // 3. +45 degrees
const float n45 = FILTER::fap2f(qq);            // 4. -45 degrees
const float ssb = p45 - n45;                    // 4. one sideband cancels
const float audio_lpf = FILTER::bwf[bw](ssb);    // 5. channel filter, 255 taps
const float audio_raw = FILTER::hpf_200f(audio_lpf); // 6. 200 Hz high-pass
const float audio_notch = notch.process(audio_raw); // 7. manual notch
const float audio_out = FILTER::jnr(audio_notch, jnr_level); // 8. noise reduction
return agc(audio_out * 32768.0f);                // 9. AGC, returns 12-bit

```

Stage 1, DC blocking. `dc1f` and `dc2f` are single-pole IIR high-passes with $k = 0.05$, giving a corner around 200 Hz at 31.25 kHz. Commented alternatives in `filter.h` offer 0.004 (below 100 Hz), 0.01 (about 100 Hz) and 0.1 (about 300 Hz).

Stage 2, the noise blanker runs on the *raw, unfiltered* I/Q, deliberately — see section 7.3. It is placed before any narrowing that would smear an impulse into something a time-domain detector cannot see.

Stages 3–4, sideband selection. This is the phasing method done in floating point. `fap1f` and `fap2f` are each a cascade of three first-order all-pass sections, arranged so that their phase difference is 90 degrees across the audio band. Subtracting one from the other cancels one sideband and doubles the other.

```

fap1f poles: k = 0.98325, 0.88497, 0.59331 -> 84 Hz, 607 Hz, 2539 Hz
fap2f poles: k = 0.07102, 0.78470, 0.94391 -> 8628 Hz, 1200 Hz, 287 Hz
Per section: y = k*(x + y_prev) - x_prev

```

The 84 Hz to 8.6 kHz pole spread is what sets the useful passband: 90-degree accuracy holds roughly from 300 Hz to 3 kHz, which is exactly why the audio low-pass and high-pass sit *after* this stage rather than before it.

It is worth pausing on the difference between this and a phasing receiver. A Tayloe rig's sideband rejection is limited by *analogue* channel matching. Here the I and Q that arrive at this code are already exactly matched — they came through identical FPGA logic — so the only error source left is the all-pass network's own phase accuracy, which is a fixed, repeatable, temperature-independent property of six float constants.

Stage 5, the channel filter is a function-pointer table indexed by the bandwidth setting:

radio.bandwidth	Function	Taps	Displayed bandwidth
-----------------	----------	------	---------------------

1	lpf_2200f_rx	255	2000 Hz
2	lpf_2400f_rx	255	2200 Hz
3 (default)	lpf_2600f_rx	255	2400 Hz
4	lpf_2800f_rx	255	2600 Hz
5	lpf_3000f_rx	255	2800 Hz

The displayed value is 200 Hz below the filter’s corner because the 200 Hz high-pass in stage 6 eats the low end. All five are 60 dB linear-phase FIRs designed at 31 250 Hz.

Stages 7–8, notch then noise reduction, then AGC. The order is deliberate and `notch.h` explains it: notching must happen *before* the AGC, because “notching after the AGC leaves the heterodyne pumping the gain and desensing the receiver.”

4.6.3 How the firmware FIRs are built

Every FIR in `filter.h` is written the same way, and it is an unusual style worth understanding because it is what makes 510 taps fit in 32 μ s.

```
#define FIR_LENGTH 256
#define __mac_tap(_h) acc += (_h)*x[i++]
#define __mac_zap(_h) i++

static const float __not_in_flash_func(lpf_2200f_rx)(const float sample)
{
    static float x[FIR_LENGTH] = { 0.0f };
    static uint8_t sample_index = 0;
    uint8_t i = sample_index;
    x[sample_index--] = sample;
    float acc = 0;
    __mac_tap(-0.000019f);
    __mac_tap( 0.000023f);
    ... // 255 of these, fully unrolled
    return acc;
}
```

Four things are going on:

1. **The delay line is 256 long with a `uint8_t` index.** The index wraps naturally modulo 256. There is no modulo operation and no branch — the wrap is free. Cost: 1024 bytes of state per FIR instance regardless of tap count.
2. **The loop is fully unrolled and the coefficients are immediate float literals in the instruction stream,** not a table in memory. No coefficient loads, no loop overhead, no index arithmetic beyond the post-increment.
3. **`__mac_zap()` skips coefficients that are exactly zero.** This is why the $f_s/4$ and $f_s/8$ filters are cheap: `lpf_fs8f_rx1` has 127 taps but only 97 real multiplies.
4. **`__not_in_flash_func()` forces the function into SRAM.** On the RP2350, code normally executes from external flash through the XIP cache; a cache miss inside the DSP path could blow the 32 μ s budget outright. Every function in the DSP path carries this attribute — 39 in `filter.h`, 14 in `dsp.h`, 2 in `cw.h`, plus `loop1()` itself, the encoder decoder, and the notch.

The worst-case SSB frame is `bwf[bw]` (255 taps) plus `hpf_200f` (255 taps) = **510 float multiply-accumulates**, plus two DC blockers, six all-pass sections, the noise blanker, two

notch biquads (about 60 cycles), up to four boxcar averages, and the AGC — all inside 8000 cycles. The firmware displays its own measured load on screen; see section 6.5.

There is also a discipline note in `notch.h` that applies to the whole DSP layer: everything is single-precision `float`. “Do not let a `double` sneak in — `double` is software emulated on the M33 and will cost you hundreds of cycles.” There is no `double` anywhere in the audio path.

4.6.4 The other demodulators

CW (`process_cw`, `dsp.h:292`) uses the same DC-block and Hilbert front end, then a single 255-tap 600–800 Hz band-pass, `bpf_700f`. It returns the AGC’d audio *and* exports the pre-AGC filtered signal by reference, which is what feeds the CW decoders. No noise reduction, no notch, no blanker.

Digital and FT8 (`process_dig`, `process_ft8`) are identical Hilbert chains ending in the 3000 Hz filter. `process_dig` returns AGC’d 12-bit audio; `process_ft8` returns the **raw float** and calls the AGC only for its S-meter side effect. Both are deliberately linear — no blanker, no notch, no noise reduction — because the decoder wants an unprocessed signal.

AM (`process_am`, `dsp.h:312`) is worth a closer look, because zero-IF envelope detection is one place where the direct-sampling architecture pays off unexpectedly. The signal is already complex, so the envelope is just the magnitude:

```
const float ii = FILTER::lpf_fs8f_rx1((float)in_i / 32768.0f); // 3906 Hz, I channel
const float qq = FILTER::lpf_fs8f_rx2((float)in_q / 32768.0f); // identical design, own
state
const float envelope = sqrtf(ii*ii + qq*qq); // one VSQRT.F32 on the
FPU
```

No diode, no detector distortion, no threshold effect. The AGC for AM is *carrier-referenced*, which is the clever part: a slow one-pole filter (`agc_alpha = 0.001f`, about a 5 Hz corner) tracks the envelope’s DC component — and at zero IF, the envelope’s DC component *is* the carrier level. Using that as the gain reference makes the AGC immune to syllabic pumping, which is the usual complaint about AM AGC. A fast-attack guard catches sudden strong signals:

```
agc_carrier += (agc_envelope - agc_carrier) * agc_alpha;
if (agc_envelope > 4.0f * agc_carrier) agc_carrier = agc_envelope * 0.5f; // within 6 dB,
immediately
```

Target output is 1600 counts at 100% modulation, leaving about 28% positive-peak headroom under the 12-bit DAC’s 2047. Note that the MBPTRX (the uDST’s Tayloe-architecture predecessor) had to run its AM detector at $f_s/4$ rather than at baseband, precisely because of the zero-IF pathologies that direct sampling removes.

4.6.5 Audio output: PWM, no codec

There is no DAC chip and no codec for receive audio. The RP2350 generates it as **two 6-bit PWM channels** summed in the analogue domain by weighted resistors:

```
pwm_set_wrap(audio_pwm, 63); // 250,000,000 / 64 = 3,906,250 Hz carrier
...
dac_audio = constrain(dac_audio, -20481, +20471);
dac_audio += 20481; // to unsigned 12-bit
dac_h = dac_audio >> 6; // GP15, the 6 MSBs
dac_l = dac_audio & 0x3f; // GP14, the 6 LSBs
pwm_set_both_levels(audio_pwm, dac_l, dac_h);
```

The carrier is **3.90625 MHz** and the sample rate 31 250 Hz with 12-bit resolution. On the board, GP14 and GP15 feed R46/R47/R51/R55 (1 k, 1 k, 1 k, 100 k) into an RC network — the schematic annotates the reconstruction corner as $f = 1/(356 \times 2\pi \times 100 \text{ nF}) = 4463 \text{ Hz}$ — and then into **U14, an LM4875** headphone amplifier on its own LM1117-regulated 5 V rail. Volume is a 10 k pot into the LM4875's DC volume-control pin, and GP16 (**MUTE**) lands on the same node to shut it down.

An earlier design revision used an external PCM1808 as an ADC; the firmware history note 0.05.250 set 250 (not using PCM1808 now) records its removal. # 5 The transmit path, end to end

Transmit is the receive path run backwards, and the symmetry is close enough that most of it can be described by reference. The genuinely new material is the microphone chain, CESSB, and the analogue power stage.

5.1 Microphone to I/Q

The microphone is sampled by the **RP2350's own SAR ADC** on channel 2 (GP28) — no external codec. One **START_ONCE** conversion is triggered per DSP frame, with the *previous* conversion read first, so mic sampling is exactly **31 250 Hz, 12 bits**, phase-locked to the FPGA frame.

On receive the microphone pin is not merely ignored, it is **actively grounded**:

```
static inline void enable_mic(void) { adc_gpio_init(PIN_MIC); adc_select_input(MIC_MUX); }
static inline void disable_mic(void) { pinMode(PIN_MIC, OUTPUT); digitalWrite(PIN_MIC, LOW); }
```

The off-chip preamplifier is **U11, an LMV321** single rail-to-rail op-amp, and its network is nicely labelled on the schematic with its own corner frequencies — all four of which check out against the component values:

Element	Components	Corner
Input high-pass	C100 10 nF into a 33.3 k Thevenin divider	478 Hz (marked "480Hz")
Feedback pole	R43 100 k in parallel with C93 390 pF	4081 Hz (marked "4 KHz")
Gain shelf	R54 10 k + C102 100 nF, OUT to -IN	159 Hz (marked "150Hz")
Output low-pass	2 k + C96 10 nF	7958 Hz (marked "8KHz")

The net effect is a non-inverting gain that rises from unity below 150 Hz to about **11 times (20.8 dB)** above it — a deliberate communications-grade shaping curve, done in analogue before the converter sees it. The electret bias comes off a separately filtered 5 V rail (5V_F, via R36 100 ohm and C94 22 µF).

DSP::process_mic() in src/dsp.h:439 then does:

```

const float ac_sig = FILTER::dcf(((float)s) * (1.0f/2048.0f)) * mic_gain;
const float mic_proc = mic_process(ac_sig, proc_level);           // soft clipper
const float mic_sig = FILTER::lpf_2600f_tx(mic_proc);           // 255-tap, 2600 Hz
float ii = FILTER::fap1f(mic_sig);                               // +45 degrees
float qq = FILTER::fap2f(mic_sig);                               // -45 degrees
if (cessb_on) cessb(ii, qq);
out_i = (int16_t)(ii * 512.0f);                                   // 10-bit to the DUC
out_q = (int16_t)(qq * 512.0f);

```

`mic_gain` is the menu setting divided by 100, range 25–200%.

Note that the same `fap1f/fap2f` all-pass pair is used here as on receive, generating the analytic (complex) representation of the microphone audio rather than resolving one. Feeding a real signal into a ± 45 -degree pair and keeping *both* outputs gives you I and Q; the DUC’s mixer then places that analytic signal at the carrier frequency, and because I and Q are exactly in quadrature by construction, the unwanted sideband and the carrier are both suppressed to arithmetic precision. That is what the README means by “mathematically perfect carrier and sideband suppression — no balanced-modulator nulling.”

5.1.1 The microphone processor

`DSP::mic_process()` is a normalised `tanh` soft clipper — normalised so that a unity input stays unity, which means changing the drive level changes the *shape* of the compression without changing the level:

```

static const float DRIVE_TABLE[6] = { 0.0f, 1.5f, 3.0f, 5.0f, 7.5f, 12.0f };
//                                     bypass subtle light typical aggressive contest
const float clipped = tanhf(sample * drive) / tanhf(drive);

```

Level 0 bypasses entirely, without touching filter state.

5.2 CESSB

Controlled-envelope SSB is a technique for raising a transmitter’s average power without raising its peak envelope power — so the PA works harder within the same headroom and the same regulatory occupied bandwidth. The idea is due to Dave Hershberger W9GR. The insight is that clipping the *audio* waveform generates splatter, but clipping the *complex envelope magnitude* and then band-limiting the result does not, because the clipping is happening on the thing the PA actually cares about.

The uDST’s implementation is twelve lines, applied to I and Q after the Hilbert pair:

```

static void __not_in_flash_func(cessb)(float& ii, float& qq)
{
    ii *= 1.5f;           // +3.52 dB of drive
    qq *= 1.5f;
    const float m2 = ii*ii + qq*qq;
    if (m2 > 1.0f)       // clip the ENVELOPE MAGNITUDE to the unit circle
    {
        const float inv = 1.0f / sqrtf(m2);
        ii *= inv;
        qq *= inv;
    }
    ii = FILTER::lpf_2600if_tx(ii); // re-band-limit, 125 taps
    qq = FILTER::lpf_2600qf_tx(qq); // identical design, independent state
}

```

The numbers:

- **Pre-clip drive of $\times 1.5 = +3.52$ dB.** That is the CESSB gain figure — average power rises about 3.5 dB for the same peak envelope, less whatever the post-filter overshoot costs.
- **Clip threshold: the unit circle, $|I + jQ| \leq 1.0$.**
- **Post-clip band-limiting: two 125-tap 2600 Hz 60 dB FIRs.** Two separate instances of the same design with identical coefficients and independent state, so I and Q stay phase-matched to the tap.

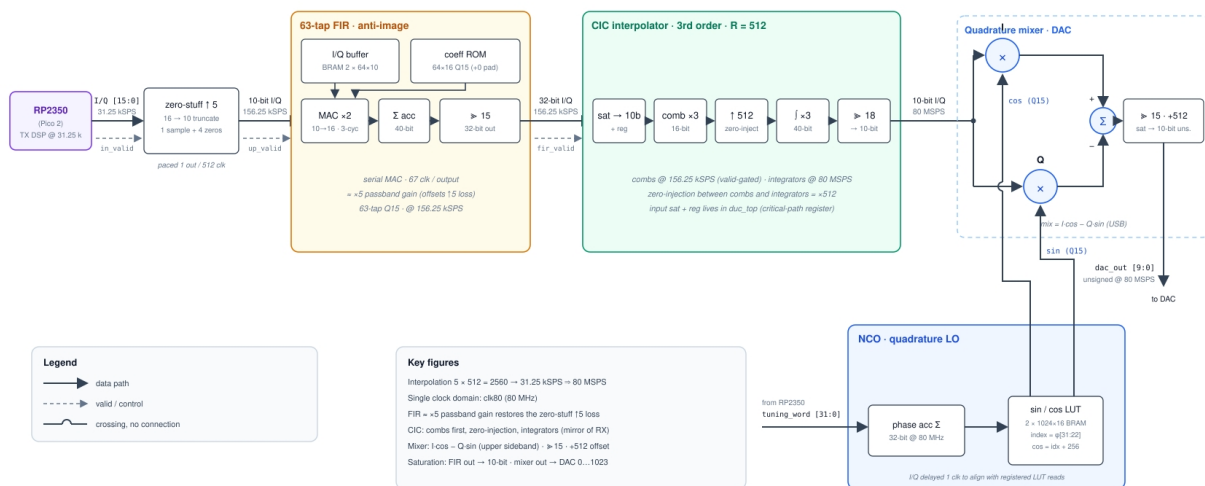
It is worth being precise about what this is and is not. Hershberger's published method uses two clip-and-filter iterations plus an explicit envelope-overshoot-control stage. The uDST does **one** iteration, so the 2600 Hz filtering after the clip regenerates some overshoot that is never re-clipped. It is a faithful implementation of the *principle* — envelope-domain clipping followed by band-limiting — reduced to what fits in the remaining cycles of a 32 μ s frame. Expect most of the benefit and some residual overshoot.

CESSB is a menu setting (`radio.cessb`) and is stored in EEPROM.

5.3 Inside the FPGA: the digital up-converter

uDST — FPGA Digital Up-Converter (TX)

DUC18.v - duc_top - single 80 MHz clock domain (clk80) - 16-bit I/Q @ 31.25 kSPS in → 10-bit DAC @ 80 MSPS out (interpolation $5 \times 512 = 2560$)



The DUC block diagram from `docs/uDST_DUC18_block_diagram.png`.

verilog/DUC18.v, and the arrangement is chosen to be **unity gain end to end**:

I/Q from the Pico (31.25 kHz, 10 bit signed, taken from the low 10 bits of the 16-bit field)

- $\rightarrow$ zero-stuff $\times 5$ $\rightarrow 156.25 \text{ kHz}$ (energy spread over 5 outputs: /5)
- $\rightarrow$ 63-tap FIR $\rightarrow 156.25 \text{ kHz}$ (about $\times 5$ passband gain: restores it)
- $\rightarrow$ 3rd-order CIC interpolator $\rightarrow 80 \text{ MHz}$ (DC gain $512^2 = 2^{18}$)
- ... SCALE_SHIFT = 18 (cancels it exactly)
- $\rightarrow$ NCO quadrature mixer, Q15 $\rightarrow 80 \text{ MHz}$ ($\gg 15$ returns to integer scale)
- $\rightarrow$ offset to unsigned, saturate $\rightarrow 10$ -bit DAC

Each stage's gain is cancelled by the next, deliberately, so 10-bit transmit samples map cleanly onto the 10-bit DAC with no scaling surprises.

5.3.1 Zero-stuffing and the interpolating FIR

Upsampling by 5 means inserting four zeros between each sample. The `upsampler_zero_stuff5_tx` module does exactly that, on a `OUT_DIV = 512` counter so the output rate is $80 \text{ MHz} / 512 = 156.25 \text{ kHz}$:

```
if (phase == 0) begin outI <= latI; outQ <= latQ; end
else begin outI <= 0; outQ <= 0; end
```

Zero-stuffing produces four spectral images; the FIR removes them. `fir_serial_tx` is 63 taps, serial (one multiplier per channel), coefficients inline in the source as a `Hmem[64]` initial block — with `Hmem[63] = 0` as the same zero-pad trick the receive FIR uses. The coefficients are Q15 with about **5 times DC gain**, deliberately, to restore the amplitude lost to zero-stuffing. The comment in `dac_top` notes the consequence: the FIR output can exceed the 10-bit range, so it is **saturated** rather than sliced.

5.3.2 The interpolating CIC

Structurally the mirror of the receive CIC: three comb stages at the low rate, then three integrators at the high rate, with zero injected between input samples:

```
if (in_valid) begin
  c0y_I <= xI - c0d_I; c0d_I <= xI;
  c1y_I <= c0y_I - c1d_I; c1d_I <= c0y_I;
  c2y_I <= c1y_I - c2d_I; c2d_I <= c1y_I;
  inj_I <= {(AW-CW){c2y_I[CW-1]}}, c2y_I; // widen, then inject
end else begin
  inj_I <= 0; // zero between samples
end
i0_I <= i0_I + inj_I; i1_I <= i1_I + i0_I; i2_I <= i2_I + i1_I;
outI <= i2_I >>> SCALE_SHIFT; // SCALE_SHIFT = 18
```

Note that an *interpolating* CIC has DC gain $R^{N-1} = 512^2 = 2^{18}$, not R^N — one factor of R less than the decimating case, because the zero-stuffing already divided by R . Hence `SCALE_SHIFT = 18` rather than 27. Comb width is 16 bits (13 is the minimum need; 16 gives three bits of margin) and the integrators are 40.

5.3.3 The mixer and the DAC

The transmit NCO is the same design as the receive one — 32-bit accumulator, top 10 bits into a 1024-entry Q15 sine table, two copies of the table for two read ports, cosine by adding 256 to the index. It is fed **the same tuning word**.

The complex mix is a single subtraction:

```
mix_q15 <= mul_i - mul_q; // I*cos - Q*sin : the upper sideband
mix_int <= mix_q15 >>> 15; // Q15 -> integer
```

$I\cos\omega t - Q\sin\omega t$ is the real part of $(I + jQ)e^{j\omega t}$ — the analytic signal translated up to the carrier. One sideband, no carrier, no image, and nothing to adjust.

Then to the DAC, which is unsigned 10-bit, so the signed result is offset by mid-scale and saturated:

```
if (mix_int > 511) dac_out <= 1023;
else if (mix_int < -512) dac_out <= 0;
else dac_out <= mix_int[9:0] + 512;
```

At top level the DAC is driven from the DUC on transmit and **forced to zero on receive**, in an output register:

```
always @(posedge clk80_int) dac_r <= mode_tx ? duc_dac : 10'd0;
```

5.4 DAC to antenna: the analogue power stage

5.4.1 The DAC

U10 is an Analog Devices AD9740ARZ: 10 bits, clocked here at 80 MHz (the part is rated well above that). Parallel single-ended CMOS input DB0–DB9 from the FPGA at 3.3 V. MODE is grounded, REFLO grounded (so the internal reference is used), and R35 = 2 k on FS_ADJ sets the full-scale output current — which, with the part’s nominal 1.2 V internal reference, works out to about **19.2 mA** (derived).

A nice pragmatic touch: the schematic notes “Bypass on pin 19, compatible with DAC900 from TI”, so C89 exists to make the footprint accept an alternative part.

5.4.2 Reconstruction filter

The DAC has two complementary current outputs, and **each** gets its own identical 7-element LC low-pass (three series inductors, four shunt capacitors), labelled “~35 MHz” on the schematic:

Leg	Series inductors	Shunt capacitors	Termination
IOUTA	L1, L2, L3 — 330 nH each	C16 100 pF, C17 180 pF, C18 180 pF, C19 100 pF	R4 49.9 ohm
IOUTB	L4, L5, L6 — 330 nH each	C26 100 pF, C27 180 pF, C28 180 pF, C29 100 pF	R16 49.9 ohm

This is the real anti-alias/reconstruction filter in the design, and it lives on the transmit side. Its job is to remove the DAC’s images above Nyquist before they reach the amplifier.

5.4.3 Driver

U3, a TI OPA2674 dual wideband current-feedback amplifier with built-in current limiting, both halves used as one inverting stage per DAC leg. Gain is $470/49.9 \approx 9.4$ times, about **19.5 dB** per half (derived from R2/R3 and R18/R19).

Both non-inverting inputs are tied together and biased from BIAS5V through R8 (10 k). BIAS5V only exists during transmit — see below — so **the drivers idle at 0 V on receive**, which is a neat way of guaranteeing they cannot leak anything into the antenna node.

Its supply comes from the 12 V rail through D1, a BZG05C3V3 — a drop keeping the OPA2674 inside its absolute-maximum total supply from a 13.8 V input. (Caveat: D1’s exact orientation could not be resolved from the schematic render; the function is clear from context but verify on the board if it matters.)

5.4.4 Power amplifier

Q1 and Q2 are Mitsubishi RD16HHF1 RF power MOSFETs in TO-220, in **push-pull**, sources grounded. This is a very well-known QRP PA device.

- **Drain feed: T4**, annotated “6 turns bifilar, FT50-43, 15 cm, 0.63 mm” (SHO-301 is a tested alternative per BOM/README.md). Pins 1 and 3 to 12 V, pin 2 to Q1’s drain, pin 4 to Q2’s drain — a 1:1 bifilar centre-feed choke that also enforces drain balance.

- **Output transformer: T3**, same core and winding spec. Primary across the two drains, secondary from the low-pass bank input to ground. 1:1, so drain-to-drain load is 50 ohms and each device sees about 12.5 ohms (derived).
- **Drain-to-gate negative feedback:** R1 470 ohm + C15 10 nF on Q1, R15 470 ohm + C25 10 nF on Q2. This is the standard broadband linearising network for this topology — it trades gain for flatness and linearity across 3.5–30 MHz.
- **Gate bias:** R22 and R23, both 1 k trimmer potentiometers fed from BIAS5V, wipers through R20 and R21 (1 k) into the gates. **These are the only two adjustments on the entire board.**

Class AB linear (inferred from adjustable gate bias, push-pull topology, drain-gate feedback, and SSB service; not stated in the repository). The supply is the **raw, unregulated 12 V input rail** taken directly at T4’s centre feed, bulk-decoupled by nine 22 µF capacitors.

Thermal: the schematic specifies heatsink 274-2AB, and the BOM lists two of them plus four TO-220 thermal insulators.

5.4.5 Transmit gating: one enable line does everything

PIN_TXEN (GP22) is a plain GPIO, and it is the single point of TX control:

1. It is the **enable pin of U4, a TLS850B0TE 5 V LDO**, whose output is the BIAS5V rail. So the PA gate bias and the driver’s input bias literally do not exist unless TXEN is high. There is no residual PA idle current on receive and no way for the driver to be biased into conduction accidentally.
2. It drives the FPGA’s tx_rx input, which flips the I/Q SPI slave’s direction and un-gates the DAC.
3. Through the ULQ2003 Darlington array it energises the T/R relay coil (TXENn).

Note the FPGA constraint file pulls tx_rx **down**, with the comment: “if the Pico hasn’t driven it yet (e.g. during its own boot), the radio defaults to RX (0) rather than risking an unintended TX.”

There is a further interlock in firmware. At boot, radio.tx_safe is set true **only if** PTT and both paddles read high — so a stuck or pressed control at power-on holds transmit off. It is not a permanent latch: switching to either digital mode sets tx_safe true unconditionally, so the interlock is clearable. Transmit is also blocked outright on the SWL band and in AM mode.

5.4.6 Output low-pass filter bank: four bands, eight relays

Each band is a 7-element (three series L, four shunt C) LC low-pass, bracketed by a pair of **Panasonic ALQ112** 12 V relays. Every inductor is specified in the schematic with its turn count and wire length — this is a hand-wound-toroid design:

Band group	Relays	Inductors	Capacitors	Control bit
80 m / 60 m	K7, K8	L23, L24, L25 — 2.2 µH, 27 turns T37-6, 40 cm	C71 820 pF, C72 1.3 nF, C73 1.3 nF, C74 820 pF	LPF1

40 m / 30 m	K5, K6	L17, L18, L19 — 1 μ H, 18 turns T37-6, 30 cm	C57 390 pF, C58 680 pF, C59 680 pF, C60 390 pF	LPF2
20 m / 17 m	K3, K4	L10 560 nH 12T, L11 620 nH 14T, L12 560 nH 12T	C47 220 pF, C48 360 pF, C49 360 pF, C50 220 pF	LPF3
15/12/10 m	K1, K2	L7 330 nH 10T, L8 360 nH 11T, L9 330 nH 10T	C34 120 pF, C35 220 pF, C36 220 pF, C37 120 pF	LPF4

Approximate cutoffs, derived from $1/(\pi\sqrt{LC_{mid}})$ and not stated anywhere in the repository: about 6, 12, 21 and 36 MHz respectively.

Relay wiring puts the common on pin 2 and grounds pin 4, so a **deselected filter has both ends grounded** — good inter-filter isolation, and the reason the deselected sections behave as harmless grounded stubs on the antenna node — true whether the radio is receiving or transmitting on one of the other bands.

Selection is again purely by tuned frequency (`set_filter()`, one-hot):

Tuned frequency	Selected
below 6 MHz	LPF1
6 to 11 MHz	LPF2
11 to 20 MHz	LPF3
20 MHz and above	LPF4

BOM/README.md carries a build warning worth repeating: “Try to get capacitors of at least 100 V for the low-pass filters.”

5.5 SWR and power metering

The same T1/T2 tandem coupler from section 4.1.1 is read while transmitting. `show_swr()` at `src/uDST.ino:1213`:

```
static const float A = 0.141570128f; // volts per ADC count
static const float B = 5.64859373f; // diode-drop / offset correction, volts

const float adcfwd = (float)(FILTER::ma1(adc_fwd_raw)); // 16-point moving average
const float adcref = (float)(FILTER::ma2(adc_ref_raw));
const float vfwd = A * adcfwd + B;
const float vref = A * adcref + (adcref > 2.0f ? B : 0.0f); // offset only if there is
real signal
if (vfwd > vref) {
    vswr = (uint32_t)(100.0f * (vfwd + vref) / (vfwd - vref)); // hundredths
    vswr = constrain(vswr, 100u, 999u); // 1.00 to 9.99
}
// power: Vpp^2 / 400, in tenths of a watt, from the UNfiltered reading for peak fidelity
const float fwd = A * (float)adc_fwd_raw + (adc_fwd_raw > 2 ? B : 0.0f);
const uint32_t po = (uint32_t)((fwd * fwd) / 400.0f * 10.0f + 0.5f);
```

Three details worth noting. The `B` offset is applied to the reflected leg **only if the reading is above 2 counts**, so a diode-drop correction is not added to what is really zero. SWR uses the *filtered* readings for stability while power output uses the *unfiltered* forward reading so that peaks read honestly. And peak hold decays at one step per 250 ms.

Full scale is `METER_PO_FS = 1000` = 10.0 W, displayed either numerically or as a 16-segment bargraph with a deliberately non-linear scale — the interpolation knots sit at 0, 1, 2, 3, 5, 7 and 10 W, with printed labels at 1, 2, 3, 5 and 10 — so that the useful low end is not squeezed into two pixels.

6 Firmware architecture

6.1 Two cores, no operating system

There is no RTOS. A grep for `xTaskCreate`, `vTaskDelay`, `QueueHandle`, `dma`, `irq_`, `attachInterrupt` or `add_repeating_timer` across the entire source tree returns **nothing**. The design is two bare-metal Arduino-Pico loops, one per core, synchronised by a hardware level from the FPGA.

	Core 0	Core 1
Entry	<code>setup()</code>	<code>setup1()</code>
Loop	<code>loop()</code>	<code>loop1()</code>
Role	UI: display, menu, spectrum FFT, PTT and keyer sequencing, SWR/power meter, FT8 slot machine and LDPC decode, EEPROM	DSP: hard-real-time 31.25 kHz sample loop
Stack	default 4 KB (shared block)	dedicated 8 KB (<code>bool core1_separate_stack = true;</code>)
Deadline	50 ms display frame, soft	32 μ s sample frame, hard

Core 1 waits at a startup barrier until core 0 finishes initialising:

```
while (!setup_complete) { tight_loop_contents(); }
```

This is not a fashionable architecture, but for this problem it is the right one. There is exactly one hard-real-time task, and it needs a whole core with nothing else on it. An RTOS would add scheduler jitter to a 32 μ s budget in exchange for services this program does not use.

6.2 The 32 μ s frame

The DSP loop is not timer-driven; it self-clocks off the FPGA's `data_ready` line on GP12, and the whole mechanism is four lines of polling (see section 4.6). Consequences worth spelling out:

- **The scheduler is a wire.** There is no timer to drift and no interrupt latency to bound.
- **Overrun degrades gracefully.** The FPGA holds the sample in a register, so if a frame runs long the result is a dropped sample, not a corrupted one.
- **Everything in the audio chain is inherently synchronous.** Mic sampling, PWM update, SPI exchange, and the encoder poll all happen exactly once per frame at exactly the same point in it.

One elegant side effect: **the rotary encoder is polled at 31.25 kHz**, because it is polled inside the DSP loop. That is roughly a thousand times faster than any human wrist, which is why it never misses a detent and needs no interrupt.

6.3 Boot-time self-checks

The firmware refuses to run in a broken configuration, and each failure has a distinct blink code from `error_stop()`:

Code	Constant	Checked
3	ERROR_SYSCLOCK	frequency_count_khz confirms 250 MHz $\pm$ 5 kHz after vreg_set_voltage(1.30 V)
4	ERROR_FPGA	data_ready edge count over 10 ms is 312–1250 (expects 625)
5	ERROR_LPF	TCA9534 GPIO expander responds on I2C
7	ERROR_FWDADC	MCP3021 at 0x48 responds
8	ERROR_REFADC	MCP3021 at 0x49 responds

There is also a conditionally compiled SPI loopback test (SPI_LOOPBACK_TEST, GP7 to GP8 through 220 ohms) that runs nine patterns including 0xDEADBEEF and 0xA5A5A5A5 and requires 14 matches in a rolling window of 16.

6.4 Cross-core communication

There are exactly two synchronisation primitives in the program:

```
auto_init_mutex(rotary_mutex);
auto_init_mutex(cw_decode_mutex);
```

No queues, no ring-buffer FIFOs, no atomics, no memory barriers. Everything else is volatile shared state plus carefully chosen handshakes:

Data	Direction	Mechanism
Encoder delta	core 1 to core 0	rotary_mutex with mutex_try_enter(..., 0) on the DSP side, so it never blocks; the local accumulator keeps the count if the lock is busy
CW decode buffer	core 1 to core 0	blocking enter on the writer, try-enter on the display side, which skips the frame if busy
I/Q for the spectrum	core 1 to core 0	volatile half-buffer handshake with a dead zone, no lock
FT8 audio	core 1 to core 0	same pattern on a 2500-float ping-pong buffer
Notch coefficients	core 0 to core 1	shadow/live double buffer gated by a volatile bool pending
radio state struct, meters, CPU counters	both ways	plain volatile

The pattern used for the bulk sample data is worth understanding because it appears twice. Core 1 writes adc_data_i[2048] and adc_data_q[2048] continuously with a power-of-two wrap. Core 0 takes whichever 1024-sample half the writer is *not* currently in — and bails out entirely if the writer is anywhere near a boundary:

```

const uint32_t sample_p = adc_sample_p;
if      (sample_p < 800)                offset = SPECTRUM_BUFFER; // take the 2nd
half
else if (sample_p > 1023 && sample_p < 1800) offset = 0;           // take the
1st half
else                                     return;                // dead zone:
skip

```

Those dead zones — 224 samples on one side (800 to 1023) and 248 on the other (1800 to 2047) — are what replaces a lock. Core 0 loses a display frame occasionally; core 1 never stalls. The FT8 version adds an `offset != last_offset` edge check so each half is consumed exactly once.

This is worth flagging as a deliberate trade rather than an oversight: **there are no memory barriers anywhere**. Correctness rests on `volatile` plus generous dead zones. On the RP2350's coherent SRAM at these rates it works, but it is the kind of thing a formal review should note.

6.5 Timing and memory discipline

6.5.1 Measured CPU load, on screen

The radio measures and displays its own DSP load. Core 1 brackets its work with `time_us_32()` and accumulates; core 0 reduces once a second:

```

cpu_usage = (100u * usage / count) >> 5; // percent of a 32 us frame

```

The `>> 5` divides by 32, so the figure is mean microseconds per frame as a percentage of the 32 μ s period. It appears in the top-right corner of the display on a red field.

Cost figures recorded in the source comments (the SPI frame time is derived from the PIO clock divider, not stated anywhere):

Item	Cost	Source
Notch (two biquads)	about 60 cycles per sample, ~0.8% of the frame	<code>notch.h</code> comment
Noise blanker	adds 64 samples (2.05 ms) of RX latency when on, zero when off	<code>dsp.h</code> comment
One 32-bit I/Q SPI frame	about 5.5 μ s, ~17% of the frame	derived
Spectrum DC estimator	64 blocks at 30.5 blocks/s = 2.1 s, a 0.5 Hz corner	<code>spectrum.h</code> comment
FT8 slot	12.64 s receive, 2.36 s decode, inside 15 s	<code>ft8ui.h</code> constants

6.5.2 Flash avoidance

On the RP2350 code normally executes from external QSPI flash through an XIP cache. A cache miss in the DSP path could blow the frame budget, so:

- Every DSP-path function carries `__not_in_flash_func()` — 39 in `filter.h`, 14 in `dsp.h`, 2 in `cw.h`, plus `loop1()`, the encoder decoder and the notch.

- Large lookup tables live in a named SRAM section via `__not_in_flash("fast_access_sram")` — the CW Gaussian envelope table, the sidetone sine table, and three utility tables.

6.5.3 Memory budget

The big consumers, all statically allocated (there is no heap use in the DSP or FT8 paths):

Buffer	Size
FT8 waterfall (<code>uint8</code>)	167 028 B
FT8 FFT scratch (<code>s_fft_re</code> , <code>s_fft_im</code>)	20 000 B
FT8 frame buffer	10 000 B
FFT temp buffer	10 000 B
FT8 audio ping-pong	10 000 B
Display sprite (240 x 135 x 2)	64 800 B
I/Q rings for the spectrum	8 192 B
Waterfall history (41 x 240)	9 840 B
Each FIR instance's delay line	1 024 B (25 instances)

Stack usage is instrumented rather than guessed: `stackpaint.h` paints a sentinel pattern (`0xC0DE57AC`) below the current stack pointer and later counts survivors to get a high-water mark. Its header carries the right warning — the watermark is monotonic, so you must exercise every deep path (menu, both CW decoders, maximum noise blanking, FT8 decode, SSB transmit with processing and CESSB, and a settings save) before trusting the number.

6.6 Settings storage

256 bytes of flash-emulated EEPROM, keyed by a magic word `0x12345678`, laid out as a flat `uint32_t` array:

Slot	Contents
0	magic key
1–4	CW dit length (ms), CW level, sidetone, CW phase
5–10	spectrum type, noise-reduction level, mic gain, CESSB on/off, bandwidth, SWR graph style
11–19	per-band spectrum reference level, 9 bands

`save_settings()` mutes the audio, waits 100 ms, writes, then unmutes — so a flash-sector write can never click through the headphones. `restore_settings()` validates and clamps **every** field against its legal range and substitutes the default if it is out of bounds, and recomputes the CW phase increment from the sidetone rather than trusting the stored copy.

Deliberately **not** persisted: frequency, band, mode, tuning step, mic processing level, noise-blanker level, notch frequency and enable, attenuator, CW decoder selection, FT8 CQ type and auto-calibration. Those all reset to defaults at power-on.

There is **no** RF, gain or frequency calibration table anywhere in the firmware. The only calibration mechanisms are the per-band spectrum reference level (a display pre-shift in 6 dB steps), the two hard-coded SWR/power scale constants, and FT8 slot-phase calibration — and none of the last three is stored. # 7 How each feature actually works

7.1 Spectrum display and waterfall

7.1.1 Why there are two FFTs in this radio

The uDST contains two completely separate FFT implementations, and confusing them is easy. They exist for different reasons.

	Panadapter	FT8
File	spectrum.h	fft2500.h
Size	1024-point complex , fixed-point <code>int16</code>	2500-point real , <code>float</code>
Input	raw I/Q at 31 250 Hz	audio at 7 812.5 Hz
Bin spacing	30.5176 Hz	3.125 Hz
Radix	radix-2, 10 stages	mixed radix 5·5·5·5·2
Origin	classic public-domain <code>fix_fft</code>	derived from KissFFT (BSD-3)

The panadapter FFT is 1024 points because it transforms the **complex** I/Q stream, so 1024 complex points at 31 250 Hz cover the full ± 15.625 kHz — exactly the DDC channel — and a power of two lets an integer radix-2 kernel run cheaply on the M33 alongside everything else. FT8’s 2500 is not a power of two, and exists only because FT8 needs 3.125 Hz bins at 7 812.5 Hz.

7.1.2 The panadapter chain

Windowing and DC removal, fused. The Hann window multiply and the DC removal are done in the same expression so there is no intermediate `int16` store:

```
const int32_t w_re = ((int32_t)ii[i] - dc1_i) * w - (int32_t)((dc1_f * wu) >> 16);
re[i] = (int16_t)((w_re + rnd) >> shift);           // shift = 15 - gain
```

The DC estimator is a Q16 leaky integrator with `DC_K = 6`, i.e. a 64-block average. At $31250/1024 = 30.5$ blocks per second that is about 2.1 seconds, a 0.5 Hz corner. The clever part is the **deviation gate**: the estimate only updates while the block mean is within one count of the estimate. That means a real carrier tuned to the centre of the display is *not* slowly absorbed into the DC estimate and notched out — it keeps reading full scale. (The source history records this as two separate fixes: “improved spectrum DC removal” and “fix spectrum bleed”.)

The transform is `fix_fft`, in-place, `int16`, with fixed per-stage scaling of `shift = 1` on all ten stages — giving an overall $1/1024$ distributed across the stages to maximise accuracy at

each step. The twiddle table is 768 `int16` entries (three quarters of a sine period, so cosine is read at an offset of $N/4$). The Q15 multiply is rounded:

```
int32_t c = ((int32_t)a * (int32_t)b) >> 14;
b = c & 0x01;          /* the last bit shifted out = the rounding bit */
a = (c >> 1) + b;
```

Magnitude avoids a square root entirely, using the alpha-max-plus-beta-min approximation:

```
return ((15u * hi) >> 4) + ((15u * lo) >> 5); // about 0.9375*max + 0.46875*min
```

Log conversion is a 4096-entry `uint8_t` lookup table returning a 5-bit result (0–31), saturating at 31. Measured from the table: above about level 11 the scale is linear at **1.667 dB per level** (20 dB per 12 levels), for a total displayed range of about **52 dB**, with a compressed toe below level 11 (levels 1–7 span only 0–12 dB).

Bin reordering puts DC in the middle, at `magnitude[511]`:

```
for (int32_t i = 0, j = 511; i < 512; i++, j--) mag[i] = log32(get_mag(re[j], im[j]));
for (int32_t i = 512, j = 1023; i < 1024; i++, j--) mag[i] = log32(get_mag(re[j], im[j]));
```

Display reduction takes only bins 33 through 992 and collapses **four bins per pixel with a max-hold** across the 240-pixel screen:

```
uint8_t droplet = magnitude[x*4+33];
droplet = max(droplet, magnitude[x*4+34]);
droplet = max(droplet, magnitude[x*4+35]);
droplet = max(droplet, magnitude[x*4+36]);
if (droplet > 31) droplet = 31;
water[wp][x] = droplet;
```

So the displayed span is 960 bins x 30.5176 Hz = **29.30 kHz** (the “30 kHz spectrum display”), at **122.07 Hz per pixel**. DC lands at $x = (511 - 33)/4 = 119.5$, which is exactly why the centre marker is drawn as two pixels at 119 and 120.

Max-hold rather than averaging is the right choice here: at 122 Hz per pixel a CW signal occupies a fraction of one pixel, and averaging four bins would bury it in three bins of noise.

There is no spectral averaging. Temporal integration comes entirely from the 41-row waterfall history and the max-of-4 spatial hold. The only level control is the per-band reference level (a pre-shift in 6 dB steps, range –4 to +5).

Rendering offers two styles: `SPECTRUM_WIND` draws a white line trace over 32 pixels of height; `SPECTRUM_GRASS` fills the whole strip with a red-to-yellow vertical gradient and then overpaints black from the top down, so the bars come out gradient-filled. The waterfall below uses a 32-entry RGB565 palette running dark red, red, orange, yellow, green, cyan, blue, with the top five entries all white.

7.2 AGC and the S-meter

The AGC is a peak follower with instant attack and exponential decay, and it solves for the gain directly rather than integrating an error:

```
static const float max_gain = 200.0f; // 46 dB ceiling
static const float k = 0.99996f;

const float magnitude = fabsf(in);
if (magnitude > agc_peak) agc_peak = magnitude; // instant attack, single sample
else agc_peak *= k;
if (agc_peak < 1.0f) return (int16_t)(in * max_gain);
const float m = 2047.0f / agc_peak; // exactly fill the 12-bit DAC
return (int16_t)(in * fminf(m, max_gain));
```

$0.99996^{31250} = e^{-1.25} \approx 0.287$, so the decay is **−10.9 dB per second**. Attack is zero — one sample. That is a fast-attack, medium-decay characteristic, appropriate for SSB.

agc_peak is also the S-meter source, and it is saved and restored around transmit so the meter does not lurch on return to receive. `DSP::smeter()` maps it through a log curve with **two calibration sets selected by frequency** (below or above 10 MHz), the lower band being less sensitive to reflect real band noise:

	Below 10 MHz	Above 10 MHz
S0 reference	10.0	2.0
S9 reference	200.0	200.0
S9+ full scale	4000.0	4000.0

The anchor is documented in the source: “S9 = −73 dBm = 141 uV PP ... need to return 10 for S9.” The display is 15 blocks, red above index 11.

7.3 Noise blanker

Time-domain, wideband, pulse-width-qualified, with interpolation replacement. It operates on the raw unfiltered I/Q, before the Hilbert pair, and only in the SSB path.

```
static constexpr uint32_t NB_LEN      = 64u; // delay line, power of two
static constexpr uint32_t MAX_BLANK_RUN = 32u; // ~1 ms at 31250; longer = signal, not impulse
static constexpr uint32_t GUARD_PRE   = 2u; // pre-ring guard (the DDC FIR smears impulses slightly)
static constexpr float alpha          = 0.002f; // ~16 ms envelope time constant

static const float thresholds[] = {
    0.0f, // level 0 - unused, returns early
    9.0f, // level 1 - only very large spikes
    7.0f, // level 2 - conservative
    5.0f, // level 3 - moderate
    3.0f, // level 4 - fairly aggressive
    2.0f, // level 5 - maximum blanking
};
```

Four design decisions in this block are each doing real work.

It runs on raw I/Q, before any filtering. An impulse is broadband by definition; narrow it and it becomes a ringing burst that no time-domain detector can pick out. This is a structural advantage of direct sampling — the firmware has access to the full 31.25 kHz-wide complex signal, not a 2.4 kHz audio channel.

Detection is L1, not L2. `amp = fabsf(I) + fabsf(Q)` — Manhattan distance, no square root. Compared against `threshold * avg_amp` where the average is a one-pole filter with a 16 ms time constant (roughly an 80 Hz corner).

Pulse-width qualification. The envelope average is updated *only* when the excursion is below threshold, **or** when an over-threshold run has already exceeded 32 samples — at which point it is declared signal rather than impulse. This is what stops the blanker from chopping the peaks off a strong nearby station: a loud voice peak lasts far longer than 1 ms, so after 32 samples the detector gives up and lets the average track it instead.

Replacement is linear interpolation, not zeroing. Zeroing a sample is itself a discontinuity, which splatters. Instead the blanker reaches back into the delay line and draws a straight line across the event in both I and Q:

```
const uint32_t ia = (s.w - gap - 1u) & NB_MASK; // the last good sample
const float stepI = (I - aI) / (float)(gap + 1u);
for (uint32_t p = 1u; p <= gap; p++)
{
    const uint32_t idx = (s.w - (gap + 1u - p)) & NB_MASK;
    s.bufI[idx] = aI + stepI * (float)p;
    s.bufQ[idx] = aQ + stepQ * (float)p;
}
```

Maximum blanked span is $32 + 2 = 34$ samples = **1.088 ms**. Cost is 64 samples (2.05 ms) of receive latency when enabled, and exactly zero when off — the function early-returns and marks the delay line inactive so it refills on re-enable.

7.4 Noise reduction (“JNR”)

It is important to be accurate here, because “noise reduction” implies more than this does. The uDST’s noise reduction is a **cascade of 8-tap boxcar moving averages** — a pure low-pass hiss reducer with no signal or noise estimation of any kind. The author’s own comment says so: “just noise reduction (not dynamic).”

```
#define JNR_FILTER_LENGTH 8

static const float jnr_maf1(const float v)
{
    static float element[8] = {0.0f};
    static uint8_t p = 0;
    static float sum = 0.0f;
    sum = sum - element[p] + v; // O(1) running sum
    element[p++] = v;
    p &= 7;
    return sum / 8.0f;
}
// jnr_maf2/3/4 are identical with independent state

static const float jnr(const float s, const uint32_t level)
{
    switch (level)
    {
        case 1: return jnr_maf1(s);
        case 2: return jnr_maf2(jnr_maf1(s));
        case 3: return jnr_maf4(jnr_maf3(jnr_maf2(jnr_maf1(s))));
    }
    return s;
}
```

Note that level 3 jumps straight from two cascades to four — there is no three-cascade option.

At 31 250 Hz a single 8-tap boxcar has its first null at 3906.25 Hz and is -3 dB at about 1720 Hz. Four in cascade give about -12 dB at 1720 Hz and a distinctly muffled passband. It is a treble roll-off with a recursive implementation, which is honest about what it is: on band hiss it takes the edge off at almost no CPU cost (one add, one subtract, one divide per stage per sample), and it does not attempt spectral subtraction, LMS, or Wiener filtering. It runs after the notch and before the AGC, and is used in the SSB and AM paths only.

7.5 Notch filter

Manual only — there is no auto-notch, no adaptive canceller, no spectral peak tracker. What it *is* is a nicely engineered twin-zero “flat bottom” notch.

The problem with a single second-order IIR notch is that it is a pinhole: a couple of Hz wide at the bottom. You cannot hand-tune onto it, and any drift takes the heterodyne out again. The uDST’s solution is to build **two** notch sections and place them either side of the target frequency, so the hump between the two nulls sits exactly at the requested depth — turning a pinhole into a tunable trough.

Design, per section:

```
const float w0 = 2.0f * pi * f0 / fs;
const float cw = cosf(w0);
float r = 1.0f - pi * bw / fs;      // pole radius sets the -3 dB width
if (r < 0.90f) r = 0.90f;
if (r > 0.9999f) r = 0.9999f;
c.g = 0.5f * (1.0f + r);          // unity gain in the passband either side
c.c2 = 2.0f * cw;
c.a1 = -2.0f * r * cw;
c.a2 = r * r;
```

And the split:

```
const float a = powf(10.0f, -depth_db / 20.0f); // linear stopband floor
const float d = 0.5f * bw * sqrtf(a);          // half the separation of the nulls
design(shadow[0], fs, f0 - d, bw);
design(shadow[1], fs, f0 + d, bw);
pending = true;
```

Runtime is two cascaded biquads, with g factored out of the numerator because $b_0 = b_2 = g$ and $b_1 = -2gc\cos\omega_0$:

```
const float t = v + s.x2 - c.c2 * s.x1;
const float y = c.g * t - c.a1 * s.y1 - c.a2 * s.y2;
```

As configured in the uDST: bandwidth 100 Hz and depth 30 dB are **fixed**; only the centre frequency is user-adjustable, at 10 Hz per encoder detent over 300–2600 Hz (default 1000 Hz). Deriving the resulting geometry from those settings:

Quantity	Value
$a = 10^{-30/20}$	0.03162
Null separation, $\pm d$	± 8.89 Hz

Pole radius r	0.98995
Passband gain g	0.99497
At least 30 dB rejection over	~26 Hz
At least 20 dB rejection over	~39 Hz
-3 dB points	±79 Hz
Cost	~60 cycles/sample, ~0.8% of the frame

The internal `set()` accepts bandwidth 20–500 Hz and depth 10–45 dB and clamps the centre frequency away from DC and Nyquist by $0.5 \text{ bw} + d + 10$, so wider or deeper settings would work if exposed.

The **cross-core coefficient handover** is a lock-free double buffer: the UI core writes `shadow[]` and sets a `volatile bool pending`; the DSP core copies `shadow` into `live` at the top of `process()` and clears the flag. State is reset on enable. And, as noted earlier, everything is `float` on purpose — a `double` here would cost hundreds of cycles on the M33.

Placement, from the header and honoured in `process_ssb`: after demodulation and the channel filter, **before** the AGC and the noise reduction.

7.6 CW transmit

7.6.1 Envelope shaping

`CW::process_cw()` is a four-state machine (`KEYUP`, `TRANSITION_TO_DOWN`, `KEYDOWN`, `TRANSITION_TO_UP`) driving a **312-entry Gaussian half-envelope table** stored in SRAM, monotonic from 32767 down to 66.

At 31 250 Hz, 312 samples is **9.984 ms of rise and fall**. Ramp-up walks the table backwards from index 311; ramp-down walks forwards from 0. A Gaussian shape is the right choice: it has no discontinuities in any derivative, so the keying sidebands fall off very fast, and 10 ms is the usual compromise between click-free and not sounding soft.

```
static const int32_t cw_gain = 100;
static const int32_t set_gain = cw_gain * 1024 / 100; // = 1024
static const int32_t max_sig = 361;
...
out_i = out_q = (((max_sig * gaussian) >> 15) * set_gain) >> 10;
```

Both I and Q carry the *same* value, which is a fixed 45-degree vector of magnitude $361\sqrt{2} \approx 511$ — a constant-phase carrier at the NCO frequency, amplitude-shaped by the Gaussian. Choosing 361 rather than 511 is what keeps $|I + jQ|$ inside the 10-bit DUC input when I equals Q.

Frequency handling: in receive, CWL adds the sidetone offset and CWU subtracts it, so the beat note lands at the sidetone pitch; in transmit the offset is zero, so the carrier is exactly on the displayed frequency.

7.6.2 Sidetone

A 1024-entry sine table driven by a 32-bit phase accumulator:

```
static const uint64_t tone = 700ull;
static const uint32_t level = 3u;
static const uint32_t shift = 8u - level; // 5
static const uint32_t phase = (uint32_t)((tone * (1ull << 32)) / SAMPLERATE);
volatile static uint32_t dds = 0;
int16_t s = 0;
if (keydown) s = dds_sin_tab[dds >> 22] >> shift;
dds += phase;
return s;
```

Amplitude is $32767 \gg 5 = 1023$, half of the 12-bit DAC's 2047. The accumulator advances even during key-up so phase is continuous across elements.

Two caveats a reader should know (see also Appendix C): this sidetone pitch is **hard-coded at 700 Hz** even though `radio.sidetone` is a menu setting (500–850 Hz in 50 Hz steps) that is stored in EEPROM — the menu setting only moves the *receive* NCO offset. And the sidetone is **hard-keyed**, with no envelope shaping, unlike the RF.

7.6.3 Keyer

`process_key()` is a blocking loop entered on paddle or PTT activity. It supports a straight key or PTT on GP0 and paddles on GP26 (dit) and GP27 (dah).

Element timing is exact by construction: each element is preceded by exactly one dit of space, and the mark ratio is 1:3.

```
// dit: cw_dit_delay(radio.cw_dit, 0) key up, then key down and cw_dit_delay(radio.cw_dit, 15)
// dah: cw_dah_delay(radio.cw_dit, 0) key up, then key down and
cw_dah_delay(radio.cw_dit*3, 15)
```

Paddle memory is implemented by sampling the opposite paddle *inside* the delay loops and latching it:

```
static void cw_dit_delay(const uint32_t ms, const uint32_t level)
{
    const uint32_t delay_time = millis() + ms;
    update_display(level);
    while (delay_time > millis())
    {
        if (digitalRead(PIN_PADB) == LOW) dah_latched = true; // check for dah
    }
}
```

with the mirror image in `cw_dah_delay()` latching `dit_latched`. There is no explicit Mode A/B menu and no dedicated iambic state machine — squeezing both paddles produces alternating elements via the two latches, which is Curtis Mode B-like behaviour.

Speed is stored as the dit length in milliseconds, not WPM: default 60 ms = 20 WPM, validated to 40–120 ms (30–10 WPM). The menu offers 10/15/20/25/30 WPM, which maps to 120/80/60/48/40 ms.

Break-in is semi-QSK with an 800 ms hang. `CW_TIMEOUT` is 800 ms; the timeout resets on every key event and the loop only exits after 800 ms of inactivity, at which point `mute()` covers the transmit-to-receive transition. This is not full QSK — there is no receive between elements. `PIN_TXEN` is raised 10 ms before the first element.

While keyed, the spectrum magnitude array is cleared and a single centre bin (`magnitude[512]`) is set to level 16, so the trace shows a one-pixel keying indicator at the tuned frequency.

7.7 CW decode: two decoders

Both decoders live in namespaces (`CWDECODE1`, `CWDECODE2`), expose the identical interface `morse_decode(const float sample)` at 31 250 Hz, and are both fed the output of the 255-tap 600–800 Hz band-pass **before** the AGC. Neither uses a Goertzel detector; both are envelope-plus-state-machine designs. The return encoding is compact: bit 7 is a live MARK flag, bits 6–0 are a one-shot 7-bit ASCII character (0x20 means word gap, 0x00 means nothing).

```
const uint8_t raw = radio.cwdecode == 1 ? CWDECODE1::morse_decode(cwsig)
                                         : CWDECODE2::morse_decode(cwsig);
```

The menu calls them “**Adaptive**” (decoder 1) and “**Schmitt**” (decoder 2). Output goes into a 20-column left-shifting ring buffer under `cw_decode_mutex`, with the last position carrying a live * key-down indicator.

7.7.1 Decoder 1: adaptive single threshold

Parameter	Value
Envelope attack / decay time constants	1 ms / 8 ms
Long-term average time constant during MARK	0.800 s
Long-term average time constant during SPACE	5.000 s
Threshold	0.50 x the long-term average
Dot/dash boundary	2.0 x dot estimate
Element/character boundary	2.5 x dot estimate
Character/word boundary	6.0 x dot estimate
Noise gate	0.3 minimum mark fraction
Dot / dash learning rates	0.25 / 0.15
Initial dot estimate	60 ms (20 WPM)
Dot clamps	35 ms to 150 ms (34 to 8 WPM)

The envelope is a rectifier plus an asymmetric one-pole. The threshold is a **single** level derived from a dual-rate long-term average — fast during MARK so it locks to the carrier level, very slow during silence so that a noise spike cannot latch the threshold high permanently (worst case recovery is about 15 s; droop over a one-second word gap is about 18%). It has an explicit noise gate that restores the saved space count, so a glitch in the middle of a gap does not corrupt the gap measurement. The dot estimator is a symmetric IIR with separate learning rates for dots and dashes (a dash contributes its duration divided by three).

7.7.2 Decoder 2: Schmitt trigger plus asymmetric estimator

The header itself documents the three differences from decoder 1.

Parameter	Value
Envelope attack / decay	1 ms / 8 ms (same)
Level tracker attack / decay	5 ms / 3.000 s, always active
Schmitt ON / OFF fractions	0.55 / 0.22 — a 2.5:1 hysteresis
Dot estimator rate when observation is shorter	0.35 (fast)
Dot estimator rate when observation is longer	0.04 (slow)
Initial dot estimate	72 ms (~17 WPM, the geometric mean of the clamps)
Boundaries and clamps	identical to decoder 1

1. A Schmitt trigger replaces the single threshold. It turns on at 55% of the tracked peak and turns off only below 22%. That eliminates chattering around the threshold and therefore eliminates the need for a noise gate at all.

2. The dot estimator is asymmetric, and this is the key robustness property:

```
static inline void md2_update_dot(md2_ctx_t* c, float obs)
{
    const float rate = (obs < c->dot_est) ? MD2_DOT_FAST_RATE : MD2_DOT_SLOW_RATE;
    c->dot_est += rate * (obs - c->dot_est);
    md2_clamp_dot(c);
}
```

The estimate converges quickly *downward* but resists drifting *upward*. That matters because decoder 1's symmetric IIR has a specific failure mode: noise or one stretched mark inflates `dot_est`, which inflates the element and character boundaries, which makes the decoder stop emitting characters — lockup. The asymmetric rate makes that essentially impossible.

3. The level tracker is always active (fast up, slow down) rather than gated on MARK.

Everything else — the IDLE/MARK/SPACE machine, the mid-space one-shot word-gap flush, the guard against emitting bare spaces, the eight-character overflow flush, the output priority of character over word space — is structurally identical between the two.

7.7.3 Morse lookup

Both use a **linear `strcmp` search over a table of dot/dash strings**, not a binary tree: 26 letters, 10 digits, and eight punctuation marks (period, comma, question mark, apostrophe, hyphen, slash, and both parentheses). An unmatched pattern returns `?`. A linear search over 44 short strings is trivially cheap at one lookup per character, and it makes the table readable in the source.

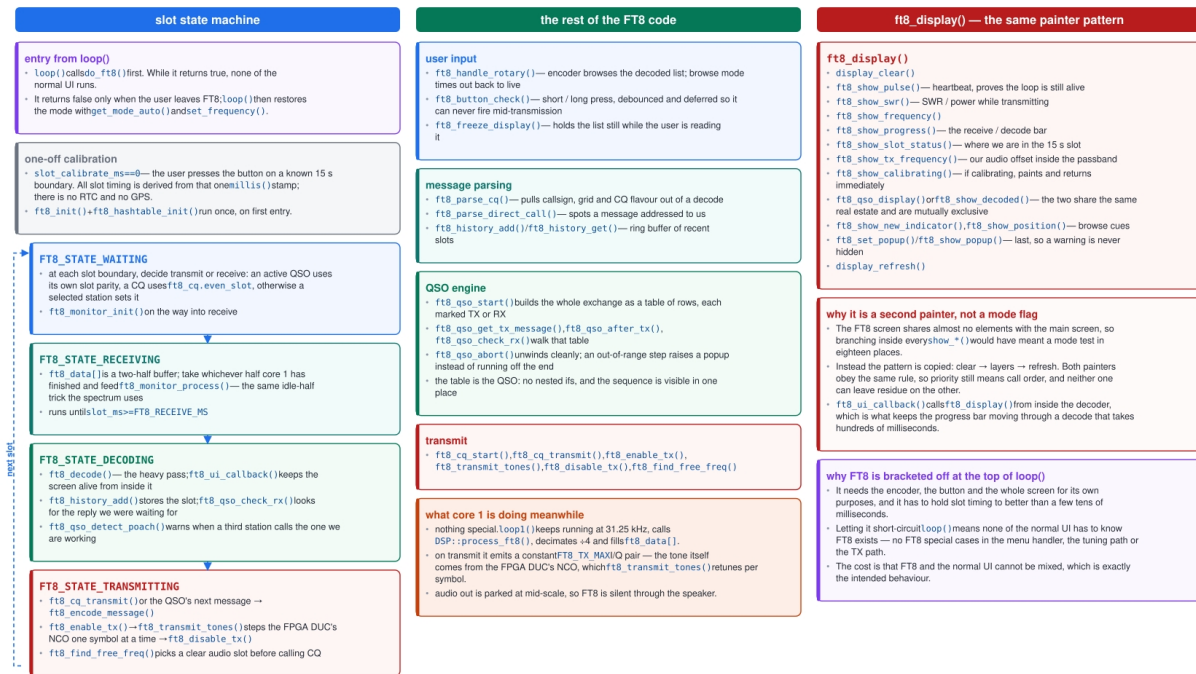
7.8 FT8

FT8 is the most substantial piece of software in the radio. It is worth understanding what it is and is not.

uDST — FT8 as a separate application

do_ft8() takes over core 0 completely. Core 1 carries on unchanged — it just fills ft8_data[] instead of driving the speaker.

Diagram 3 of 3



Ian Mitchell VK7IAN - uDST direct-sampling transceiver - generated from uDST_12.ino

The FT8 program structure, from docs/uDST_structure_3_ft8.png.

7.8.1 Provenance

ft8.h is a restructured single-header port of kgoba/ft8_1ib by **Karlis Goba (MIT licence)** — the MIT notice is retained in the file. It is *not* a port of the WSJT-X Fortran. The listed modifications: FT4 removed, KissFFT replaced by the fixed-size fft2500.h, no heap allocation anywhere, no runtime configuration struct, and the LDPC working arrays made static (which keeps about 115 KB off the stack).

7.8.2 Protocol constants

Constant	Value	Meaning
FT8_SYMBOL_PERIOD	0.160 s	160 ms per symbol, so 6.25 baud
FT8_SLOT_TIME	15.0 s	slot length
FT8_BLOCK_SIZE	1250	real samples per symbol at 7812.5 Hz
FT8_FFT_SIZE	2500	FFT length (0.32 s of audio)
FT8_TIME_OSR	2	two FFTs per symbol, 80 ms apart
FT8_FREQ_OSR	2	3.125 Hz bins, half the tone spacing
FT8_MIN_BIN / FT8_MAX_BIN	32 / 481	200 Hz to 3000 Hz

FT8_WF_NUM_BINS	449	usable bins
FT8_MAX_BLOCKS	93	15.0 / 0.160
FT8_WATERFALL_SIZE	167 028	93 x 1796 bytes
FT8_MAX_CANDIDATES	80	sync candidates evaluated
FT8_LDPC_ITERATIONS	20	belief-propagation iterations
FT8_MIN_SCORE	10	sync-score floor
FT8_ND / FT8_NN	58 / 79	data symbols / total symbols
FTX_LDPC_N/K/M	174 / 91 / 83	the (174,91) LDPC code
FT8_CRC_POLYNOMIAL	0x2757	CRC-14
FT8_MAX_DECODED	50	messages retained per slot

The Costas sync array is {3, 1, 4, 0, 6, 5, 2}, repeated at symbol positions 0–6, 36–42 and 72–78. The Gray map is {0, 1, 3, 2, 5, 6, 4, 7}.

7.8.3 Audio front end

DSP::process_ft8() is the standard USB chain minus AGC gain: DC block, the ± 45 -degree Hilbert pair, subtract, then the 255-tap 3000 Hz low-pass, returning a raw float. The AGC is still *called* so the S-meter works, but its gain is not applied.

Decimation from 31 250 Hz to 7 812.5 Hz is a **plain 1-in-4 decimator** — the 3 kHz FIR is the anti-alias filter:

```
volatile static struct { uint32_t c : 2; } downsample = { 0 };
if (downsample.c == 0)
{
    // note, ft8_data is two buffers of FT8_FFT_SIZE/2 values
    ft8_data[ft8_sample_p] = ft8_audio;
    ft8_sample_p = (ft8_sample_p + 1) % FT8_FFT_SIZE;
}
downsample.c++;
```

Note the margin: the new Nyquist is 3906.25 Hz and the filter corner is 3000 Hz, leaving 906 Hz. Adequate, but not generous.

ft8_data is used as two 1250-sample ping-pong halves. Core 0 consumes whichever half just completed and calls ft8_monitor_process() once per 1250 samples — that is, once per 160 ms symbol.

7.8.4 Building the waterfall

A sliding 2500-sample (0.32 s) analysis frame, advanced by 625 samples per time-sub, two subs per block:

```

for (int time_sub = 0; time_sub < FT8_TIME_OSR; ++time_sub)
{
    // slide the frame forward by one subblock
    for (int pos = 0; pos < FT8_FFT_SIZE - FT8_SUBBLOCK_SIZE; ++pos)
        me->last_frame[pos] = me->last_frame[pos + FT8_SUBBLOCK_SIZE];
    for (int pos = FT8_FFT_SIZE - FT8_SUBBLOCK_SIZE; pos < FT8_FFT_SIZE; ++pos)
        me->last_frame[pos] = frame[frame_pos++];

    for (int pos = 0; pos < FT8_FFT_SIZE; ++pos)
    { s_fft_re[pos] = hann_window[pos] * me->last_frame[pos]; s_fft_im[pos] = 0.0f; }
    ft8_fft_forward(s_fft_re, s_fft_im);

    for (int freq_sub = 0; freq_sub < FT8_FREQ_OSR; ++freq_sub)
        for (int bin = FT8_MIN_BIN; bin < FT8_MAX_BIN; ++bin)
        {
            const int src = bin * FT8_FREQ_OSR + freq_sub;
            const float db = log10e * logf(1E-12f + re*re + im*im);
            const int sc = (int)(2.0f * db + 240.0f);
            me->wf.mag[offset++] = (sc < 0) ? 0 : ((sc > 255) ? 255 : (uint8_t)sc);
        }
}

```

The important geometric point: the analysis frame is **twice the symbol length**, which is what yields 3.125 Hz bins — half the 6.25 Hz tone spacing. (The source comment calls this “2x zero-padded”, which is a misnomer; nothing is zero-padded.) Half-bin frequency resolution and half-symbol time resolution together are what let the sync search find a signal without being on its exact grid.

Storage is `uint8` at **0.5 dB per LSB with a –120 dB offset**, so the whole 93 x 1796 byte waterfall fits in 167 KB and every magnitude comparison downstream is an integer subtraction.

The window is a Hann (sin-squared) window with the 2/N FFT normalisation **baked into the table**, which is why the `FT8_FFT_NORM` constant is never applied separately:

```

const float x = sinf((float)M_PI * k / N_FFT);
const float hann = (2.0f / (float)N_FFT) * x * x;

```

7.8.5 The 2500-point FFT

`fft2500.h` is derived from KissFFT with every twiddle precomputed as a `const` flash table — no initialisation, no heap. It is a **real-input** FFT done the standard way: pack the real pairs as complex, run an N/2 complex FFT, then recombine.

1250 factors as **5•5•5•5•2**, so the factor table is {5,250, 5,50, 5,10, 5,2, 2,1, 0,0} and the butterflies needed are radix-5 and radix-2. Costs: 15 000 bytes of twiddles in flash, 10 KB of temp buffer in BSS.

7.8.6 Sync search: finding the signal

`ft8_sync_score()` is a **differential** Costas correlator. For each of the 21 sync symbols it accumulates the expected tone’s magnitude minus its frequency neighbours and minus its time neighbours, then normalises by the count:

```

if (sm > 0) { score += MAG(p[sm]) - MAG(p[sm - 1]); ++nav; }
if (sm < 7) { score += MAG(p[sm]) - MAG(p[sm + 1]); ++nav; }
if (k > 0 && ba > 0) { score += MAG(p[sm]) - MAG(p[sm - FT8_BLOCK_STRIDE]); ++nav; }
if ((k+1) < FT8_LENGTH_SYNC && ...) { score += MAG(p[sm]) - MAG(p[sm + FT8_BLOCK_STRIDE]); ++nav; }

```

Taking *differences* rather than absolute magnitudes is what makes the metric immune to a rising noise floor or a nearby strong carrier — it responds to the *contrast* of the Costas pattern, not its level.

Then the search brute-forces the whole grid:

```
for (cand.time_sub = 0; cand.time_sub < FT8_TIME_OSR; ++cand.time_sub)
  for (cand.freq_sub = 0; cand.freq_sub < FT8_FREQ_OSR; ++cand.freq_sub)
    for (cand.time_offset = -10; cand.time_offset < 20; ++cand.time_offset)
      for (cand.freq_offset = 0; (cand.freq_offset+7) < FT8_WF_NUM_BINS; ++cand.freq_offset)
```

That is $2 \times 2 \times 30 \times 442 = 53\,040$ sync evaluations per slot. The time window is -1.6 s to $+3.04$ s (30 steps of 160 ms) and the frequency window is 200–2956 Hz on a 3.125 Hz lattice. Candidates scoring above 10 go into an 80-entry **min-heap**, then get heap-sorted into descending score order.

That 4.72 s time-search window is what makes the radio usable without a GPS or NTP clock — see “Slot timing” below.

7.8.7 Soft demapping and LDPC

`ft8_extract_symbol()` is a max-log 8-FSK Gray demapper producing three bit log-likelihoods per symbol:

```
for (int j = 0; j < 8; ++j) s[j] = MAG(wf[kFT8_Gray_map[j]]);
logl[0] = max4(s[4],s[5],s[6],s[7]) - max4(s[0],s[1],s[2],s[3]);
logl[1] = max4(s[2],s[3],s[6],s[7]) - max4(s[0],s[1],s[4],s[5]);
logl[2] = max4(s[1],s[3],s[5],s[7]) - max4(s[0],s[2],s[4],s[6]);
```

The 174 LLRs are then rescaled so their sample **variance equals 24** — the LDPC decoder wants its inputs on a known scale, and this is how it gets one without knowing the absolute signal level.

`bp_decode()` is **sum-product belief propagation** over the (174, 91) code: column weight 3, row weight 6 or 7, 20 iterations maximum, with static working arrays. Notably the non-linearities are Padé rational approximations rather than library calls — there is no `tanhf()` in the inner loop:

```
static const float fast_tanh(float x)
{
  if (x < -4.97f) return -1.0f;
  if (x > 4.97f) return 1.0f;
  const float x2 = x * x;
  return x * (945.0f + x2*(105.0f + x2)) / (945.0f + x2*(420.0f + x2*15.0f));
}
```

A hard decision and a parity check run every iteration, with early exit on zero errors.

7.8.8 Acceptance and duplicate suppression

CRC-14 is computed over $96 - 14 = 82$ bits with the CRC field zeroed, polynomial 0x2757. A candidate is accepted **only if the LDPC decoder converged to zero errors AND the CRC matches**. There is no partial acceptance.

Duplicates are suppressed with a 50-slot open-addressed hash table keyed on the 14-bit CRC, comparing both the hash and the payload bytes with linear probing on a clash. Callsign-hash entries are aged out after 10 slots.

7.8.9 What is deliberately absent

There is **no subtractive decoding**, no signal re-synthesis-and-subtract, no ordered-statistics decoding, and no second decoding pass. `ft8_decode()` makes one linear sweep over the sorted candidate list. Separation of overlapping signals therefore rests entirely on the 3.125 Hz bin resolution and the differential sync metric.

This is the honest trade for running on a 250 MHz microcontroller inside a 2.36 second decode window alongside a live radio. Expect fewer decodes per slot than WSJT-X on a PC, particularly in crowded conditions.

7.8.10 SNR estimate

The noise floor is the **25th percentile** of the entire waterfall, found with a 256-bin histogram. Signal is the mean Costas-tone magnitude for that candidate. Then:

```
const float sig_db = (float)sig_sc * 0.5f - 120.0f;
const float noise_db = (float)noise_floor * 0.5f - 120.0f;
// SNR per bin, normalised to the 2500 Hz reference BW: 10*log10(2500/3.125) = 29 dB
const float snr = sig_db - noise_db - 29.0f;
```

Using a percentile rather than a mean is what makes the noise estimate robust when the band is full of signals. Output lines are formatted `"%+05.1f %+4.2f %4.0f %s"` — SNR, time offset, audio frequency, text.

7.8.11 Message packing

The payload is the standard FT8 77-bit structure in ten bytes plus a cached hash. Type is read from `i3 = (payload[9]>>3)&0x07` with the `n3` subtype assembled from two fields. All ten message types are enumerated (standard, non-standard call, free text, telemetry, EU VHF, ARRL Field Day, ARRL RTTY Roundup, DXpedition, contesting, WWROF), plus an unknown catch-all.

The packers are the standard FT8 field arithmetic:

- `pack_basecall()` — six-character base call to $i_0 \cdot 36 \cdot 10 \cdot 27^3 + \dots$, with the 3DA0-to-“3D0” and 3X-to-“Q” special cases.
- `pack28()` — the tokens `DE=0`, `QRZ=1`, `CQ=2`, then `CQ nnn/CQ AAAA` as 3+v, then either a 22-bit hash or a base-call encoding.
- `save_callsign()` — the 22-bit callsign hash: base-38 over 11 characters, then `n22 = ((47055833459 * n58) >> 42) & 0x3FFFFFF`. The derived 12- and 10-bit hashes are just right-shifts. Stored in a 256-entry table indexed `(hash10 * 23) % 256` with linear probing.
- `packgrid()` — `MAXGRID4 = 32400`, then +1 blank, +2 RRR, +3 RR73, +4 73, else reports as `MAXGRID4 + 35 + dB`.
- Free text — 13 characters in the 42-symbol alphabet (space, 0-9, A-Z, and `+ - . / ?`), packed base-42 into nine bytes.

`ftx_message_encode()` tries standard, then non-standard call, then falls back to free text.

7.8.12 Encoding and transmission

`encode174()` multiplies the 91-bit message by the bit-packed generator matrix using a per-byte parity function, appending 83 parity bits. `ft8_encode()` then lays out **S7 D29 S7 D29 S7** — 79 symbols, Costas at 0–6, 36–42, 72–78, the other 58 positions taking three codeword bits each through the Gray map.

Now the part that is genuinely distinctive about this radio. There is no audio-rate tone generator. In FT8 mode the Pico feeds the DUC a *constant DC I/Q vector*:

```
case MODE_FT8: tx_i = tx_q = FT8_TX_MAX; break; // FT8_TX_MAX = 360
```

and the tone is created by **re-programming the FPGA NCO once per symbol**:

```
absolute_time_t tone_process_next = make_timeout_time_ms(FT8_SYMBOL_MS);
for (int i = 0; i < FT8_NN; i++)
{
    const float tone = (float)tones[i] * FT8_TONE_SPACING; // 6.25 Hz
    const uint32_t offset_100hz = (uint32_t)((audio_base_hz + tone) * 100.0f);
    set_frequency(offset_100hz);
    ...
    busy_wait_until(tone_process_next);
    tone_process_next = delayed_by_ms(tone_process_next, FT8_SYMBOL_MS);
}
```

Since the NCO resolution is 0.01863 Hz, the 6.25 Hz tone grid is essentially exact.

The consequences are worth being precise about, in both directions. **In favour:** the envelope is perfectly constant — there is no amplitude modulation at all — which is ideal for the PA and gives the cleanest possible signal in the amplitude domain. **Against:** the symbol transitions are abrupt phase/frequency steps. This is **unshaped MFSK, not the GFSK/raised-cosine ramp that WSJT-X generates**, so the keying sidebands are wider than a WSJT-X-generated signal of the same power. Total transmission is 79 x 160 ms = 12.64 s, and a long press aborts mid-transmission.

7.8.13 Slot timing without a clock

There is **no GPS and no NTP**. FT8 needs to know where the 15-second boundaries are, and the uDST gets there two ways.

Manual. The user presses the encoder button at a known 15-second boundary and `slot_calibrate_ms = millis()`.

Automatic (`ft8_auto_calibrate()`), which is a neat piece of bootstrapping. Start decoding at an arbitrary phase. Because the sync search covers -1.6 s to +3.04 s, decodes will still succeed if you are within that window — they will just all report a consistent time offset. So: take the **median** of all decodes in the slot (via insertion sort) and apply it as the correction. If a slot yields nothing, advance by `FT8_CAL_STEP_MS = 4000 ms` — chosen specifically because 4 s steps overlap the 4.72 s search window, so no phase can be missed — and retry up to five times before reporting “No signals found”. Transmit is inhibited throughout calibration. (The 4.72 s figure is the source’s own, from an `ft8ui.h` comment; the loop bounds in `ft8.h` actually give 30 steps of 160 ms = 4.80 s of offsets spanning -1.6 s to +3.04 s, i.e. 4.64 s between the extreme candidate positions. Either way the 4 s step overlaps.)

The state machine is `WAITING -> RECEIVING (12.64 s) -> DECODING (~2.36 s) -> WAITING, or WAITING -> TRANSMITTING`, driven from core 0 with `slot_ms = (millis() - slot_calibrate_ms) % 15000`.

To keep the display responsive during the decode, `ft8_decode()` takes an optional callback invoked every 16 candidates.

7.8.14 The user interface

Decoded lines go into a **200-entry ring buffer** (about four slots' worth at 50 per slot), each tagged with whether it arrived in an even slot and with a low-order slot number for age colouring. The display shows 12 lines truncated at 38 columns. Turning the encoder freezes a snapshot for browsing (10 s timeout); a button press selects a line, which arms the QSO response.

`ft8_qso_start()` builds a five-row (calling someone directly) or six-row (answering a CQ) exchange with per-row state colouring — pending, current, waiting, retry, timeout, done — up to three retries per row. It validates the response by trial-encoding row 1 before committing to it. It also detects being “poached” and warns with `CLASH` if a third station is within 60 Hz.

`ft8_find_free_freq()` picks a CQ frequency properly rather than guessing: it scans the last four waterfall blocks over bins 96–800 (300–2500 Hz), takes a per-bin peak, and finds the longest run of bins below `noise_floor + 30` (which is +15 dB, since one LSB is 0.5 dB). It requires 24 consecutive bins (75 Hz) and returns the centre of the run. CQ types offered are CQ, CQ DX, CQ WWFF, CQ POTA and CQ SOTA.

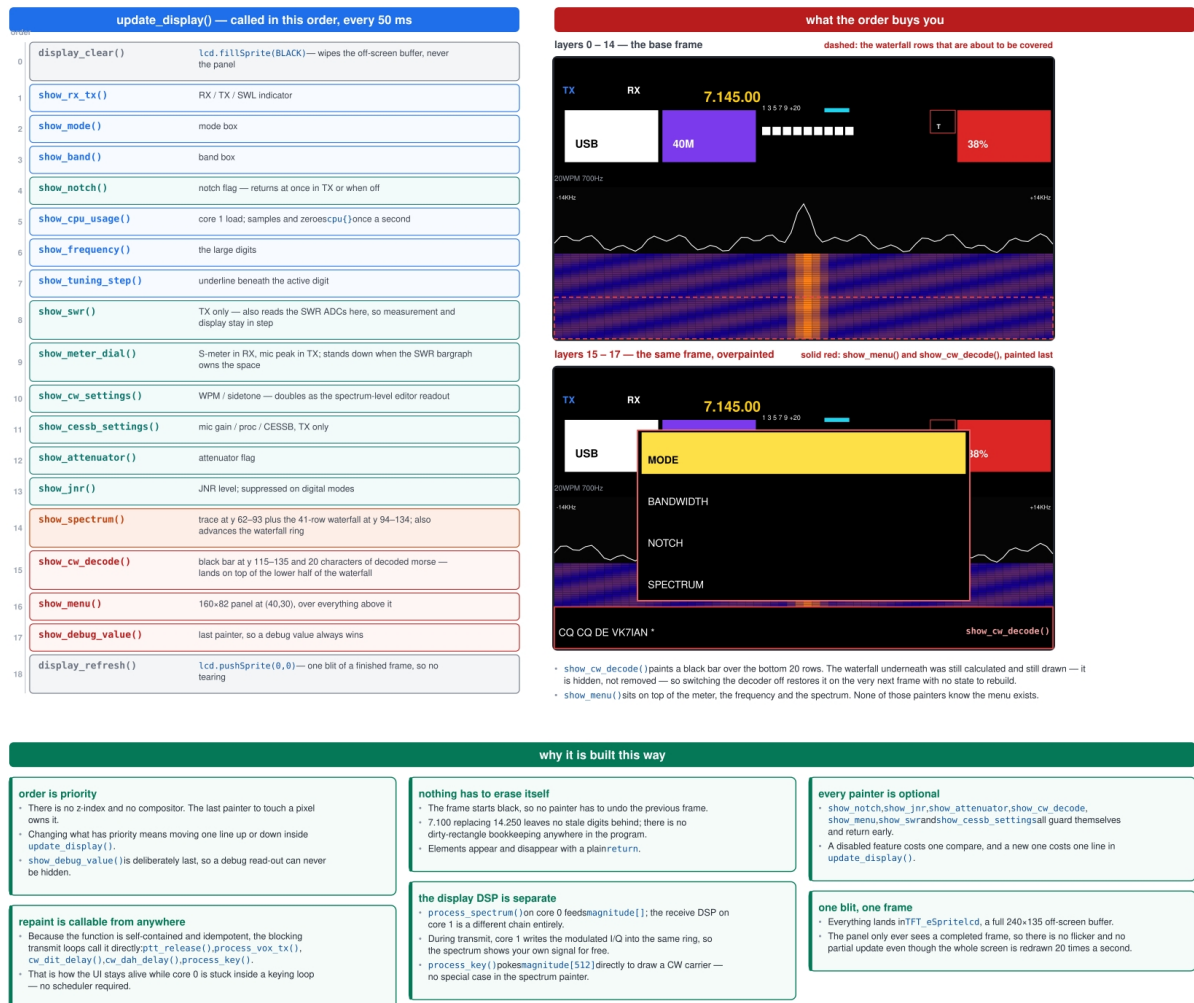
The FT8 receive level bar uses a bent log scale over 0–90 dB across 239 pixels, with the knee placed so that S9 (46.02 dB) lands at 80% of full width.

7.9 Display and user interface

uDST — update_display() and the painter's algorithm

Every frame is built from nothing, in a fixed order, into an off-screen sprite. Call order is the only priority mechanism in the program.

Diagram 2 of 3



Ian Mitchell VK7IAN - uDST direct-sampling transceiver - generated from uDST_12.ino

The display program structure, from docs/uDST_structure_2_display.png.

The display is an **ST7789-based 240 x 135 IPS TFT** (Adafruit 4383) on SPI0 at 20 MHz, four-wire, with the backlight hard-on and the on-board microSD slot unused.

Rendering is **full-frame into an off-screen sprite followed by a single blit**:

```
lcd.createSprite(LCD_WIDTH, LCD_HEIGHT); // 240 x 135 x 2 = 64,800 bytes
...
display_clear() -> lcd.fillSprite(LCD_BLACK)
display_refresh() -> lcd.pushSprite(0, 0)
```

update_display() composites 18 layers in a fixed order and pushes, at **20 frames per second**. Drawing into a sprite rather than the panel is what makes the display tear-free without any synchronisation to the panel's own refresh.

A small piece of craft in `util.h`: a compile-time bit-reverse macro chain (`_rev2` / `_rev4` / `_rev8` / `_rev16`) is used to byte- and bit-swap all the colour constants **once at compile time**, so no runtime byte-swapping is needed when pushing pixels.

The **encoder** uses Ben Buxton's state-table decoder in half-step mode, polled inside the DSP loop at 31.25 kHz. The **only panel button** is the encoder's own pushbutton: a short press cycles the tuning step by a factor of ten (100 Hz to 10 kHz), a long press (800 ms) opens the menu. Other inputs are PTT on the microphone jack ring, the CW paddle jack, and an analogue volume pot wired directly to the audio amplifier's DC volume-control pin.

The **menu** has 20 top-level items and about 90 option leaves, with a 5 second timeout. Popups also time out after 5 seconds.

Auto mode selection deserves a mention because it is a nice usability touch — the radio picks the mode from where you are tuned:

Frequency	Mode
SWL band	AM
7.074 MHz, 3.573 MHz	USB (the FT8 watering holes)
7.000–7.060 MHz	CWL
14.000–14.060 MHz	CWU
below 10 MHz	LSB
otherwise	USB

8 Hardware reference

8.1 The FPGA

U8 is a BRS-100-GW1NR9 module from Brisbane Silicon — a 36-pin castellated board, not a bare chip. The device on it is a **Gowin GW1NR-LV9QN88PC6/I5** (GW1NR-9C, QN88 package).

Build notes from `verilog/README.md`:

- Create a Gowin project with device `GW1NR-LV9QN88PC6/I5`.
- In Place and Route, under Dual-Purpose Pin, check **“Use SSPI as regular IO”**.
- Program with `openFPGALoader -b tangnano9k -f uDST.fs` — the module presents a Tang Nano 9K compatible programming interface. Bitstreams are committed to the repository.

Clock. The stock 27 MHz oscillator on the module is **replaced with a 24 MHz TCXO**, Mouser part `FT3MHUPM24.0-T1`. This is why the oscillator does not appear in the uDST BOM — it lives on the module. No jitter or phase-noise specification appears anywhere in the repository; the README’s argument for the architecture is qualitative on this point.

PLL. One Gowin `rPLL` hard block (the module filename `p11_20_80.v` is legacy):

```
FCLKIN    = "24"
IDIV_SEL  = 2      // divide by 3
FBDIV_SEL = 9      // multiply by 10
ODIV_SEL  = 8
CLKFB_SEL = "internal"
```

$24 \times 10/3 = 80.000$ MHz exactly, with a 640 MHz VCO. Commented-out alternatives for 20 MHz and 27 MHz inputs are retained in the file. The internal reset is held until `LOCK` asserts. Only `CLKOUT` is used.

`clk80` is forwarded to both converters through an `ODDR` primitive with `D0 = 1, D1 = 0` — using a DDR output buffer as a clock forwarder is standard practice, because it gives a clean output path with matched delay rather than routing a clock through general fabric.

I/O standards (from `radio_top.cst`, explicit): all 32 GPIO — both converter buses, `clk80`, `tx_rx`, both SPI buses and `data_ready` — are **LVC MOS33**. `sys_clk` is LVC MOS33 with a pull-up. The six status LEDs are **LVC MOS18** on the module’s own separate 1.8 V VCCIO bank and are not wired on the uDST board.

Two pull-mode decisions in the constraints file carry safety weight and are commented as such: `tx_rx` is pulled **down**, so an undriven line means receive; both chip selects are pulled **up**.

There is also a routing note explaining the bus assignment: each parallel bus is kept **entirely within one header** (J5 or J6) to avoid splitting it across what look like opposite package edges.

Resource utilisation is not stated anywhere in the repository. What the design actually uses: a 255-tap FIR with 256-word BRAM buffers and two serial multipliers; a

3rd-order CIC with 48-bit accumulators; a 63-tap TX FIR; a 3-stage interpolating CIC; and four copies of a 1024-entry sine LUT (two per NCO, for dual read ports).

8.2 The MCU

U13 is a Raspberry Pi Pico 2 (castellated) — RP2350, dual Cortex-M33 — mounted on the **bottom** side of the board and soldered by its castellations.

Item	Value
Clock	250 MHz, verified at boot to ± 5 kHz
Core voltage	1.30 V, set explicitly via <code>vreg_set_voltage()</code>
Flash	4 MB, no filesystem
PSRAM	none
Toolchain	Arduino-Pico core (Earle Philhower), <code>-03</code> , USB stack disabled
SMPS mode	GP23 driven high to force PWM (low-noise) mode

That last item is a small but real design detail: the Pico's on-board buck converter defaults to a pulse-skipping mode at light load, which produces variable-frequency switching noise. Forcing continuous PWM mode costs a little efficiency and buys a fixed, predictable switching frequency — which matters when there is a 12-bit converter with an antenna on it a few centimetres away.

Both cores are used. Core 1 gets a **dedicated 8 KB stack** (`bool core1_separate_stack = true;`).

8.3 Power

Input: a barrel jack (J5, PJ-002AH), nominal 12 V. **No input voltage range is stated anywhere in the repository.** Component ratings bound it: the 22 μ F bulk capacitors are 25 V, the MC7805 tolerates 35 V in, and the P-channel FET is a 60 V part. Practically 12–13.8 V.

Protection:

- **T6, a Pulse PA4339.501** four-terminal common-mode choke / EMI filter, with 100 nF on each side.
- **Q3, an FQD11P06TM** P-channel MOSFET for **reverse-polarity protection**, gate set by a 10 k / 10 k divider giving about –6 V of gate-source drive at 12 V input (fully on) and blocking on reverse.
- **There is no fuse or polyfuse** anywhere in the BOM or schematic. Worth knowing.

Rails:

Rail	Source	Feeds
12 V	input via T6 and Q3	PA drains (T4 centre feed), all relay coils, OPA2674 supply via D1, LDO inputs. Bulk: nine 22 μ F plus more

5 V	U18, MC7805CD2TR4G , fed through R59 (10 ohm, 2.4 W)	FPGA module (via D6 SS14), Pico VSYS (via D7 SS14), the CBT3253 muxes, MMIC bias, mux bias divider, LCD (via 10 ohm), mic rail (via 100 ohm), audio LDO (via 10 ohm)
BIAS5V	U4, TLS850B0TE , a 5 V LDO with an enable pin tied to TXEN	PA gate bias trimmers and the OPA2674 input bias — transmit only
5VF	5 V through 100 ohm and 22 μ F	microphone preamp and electret bias
Audio 5 V	U12, LM1117MP-5.0 , fed through 10 ohm	LM4875 only — a dedicated clean rail
3V	Pico's own 3V3_OUT	TCA9534, both MCP3021s, all Pico-side pull-ups, ADC_VREF
3v3	BRS-100 module pin 19	AD9236 AVDD/DRVDD, AD9740 AVDD/DVDD, ADC common-mode bias

Note the last two rows: **3v** and **3v3** are **separate nets**. It is an easy thing to conflate on a schematic, and the split is deliberate — the converters run off the FPGA module's regulator so their logic levels track the FPGA's VCCIO, while the control logic runs off the Pico's.

Note also the number of small series resistors feeding sub-rails (10 ohm to the LCD, 10 ohm to the audio LDO, 100 ohm to the microphone rail): these are RC isolation, not accidental. For the same reason the audio amplifier gets its own regulator rather than sharing the main 5 V rail.

BOM/README.md: "All SM resistors are 1206 thick film of at least 125 mW except R59 which should be at least 2.4 W." The Rev 1.1 changelog records "change R59 to 2.4 W" — presumably learned the hard way.

Current draw is not documented anywhere in the repository.

8.4 Support ICs

Device	Ref	Role
TCA9534DWR	U17	8-bit I2C GPIO expander at 0x20 (A0/A1/A2 grounded). Drives P0–P3 = LPF1–4, P4 = attenuator (all through the ULQ2003), and P5/P6 = HPF1/HPF2 directly as the mux select lines. P7 unused, INT# unconnected
ULQ2003D1013	U16	7-channel Darlington sink array, COM tied to 12 V for coil flyback clamping. IN1–5 from the expander, IN6 from TXEN ; OUT6 is the T/R relay coil
MCP3021A0T	U1	10-bit I2C ADC at 0x48, reads forward power
MCP3021A1T	U2	10-bit I2C ADC at 0x49, reads reflected power

OPA2674IDRG4	U3	Dual wideband CFB op-amp: the TX driver. The only op-amp in the RF path
LMV321SN3T1G	U11	Microphone preamplifier
LM4875MX	U14	Headphone amplifier with DC volume control and headphone sense
SN74CBT3253 x2	U5, U6	Dual 4:1 FET muxes selecting the receive high-pass bank
TLS850B0TE	U4	TX-gated 5 V LDO for PA bias
LM1117MP-5.0	U12	Audio amplifier's own 5 V rail
MC7805CD2TR4G	U18	Main 5 V regulator

Note the driver arrangement for the expander: relay coils go through the Darlington array (which needs the current and provides the flyback clamp), but the two mux select lines are driven **straight from the expander**, because they are CMOS logic inputs. Two different loads, two different treatments, on the same eight-bit port.

The **audio output** is headphone-only — VO1 and VO2 to a 3.5 mm jack with HP SENSE wired for automatic bridged-to-single-ended switching. There is no speaker connector and no speaker driver.

8.5 PCB

Revision 1.1, exported from EasyEDA v6.5.57.

Property	Value
Copper layers	6
Board size	120.13 x 160.02 mm, with a 33.02 x 6.35 mm notch in the bottom edge
Enclosure	Hammond 1455P1601BK (160 x 125 mm profile) — the board slides into the extrusion's side grooves
Front/back panels	separate 2-layer boards, 125.0 x 30.0 mm, 4 mm corner radii
Display window	26.5 x 16.0 mm in the front panel — exactly the 1.14 inch active area

Layer usage is the notable part, and it can be read directly out of the Gerbers by counting draw apertures per layer:

- **Inner 1 and Inner 4: pure copper pours.** Zero trace segments. 283 filled regions each (pours with thermal reliefs and via clearances).
- **Inner 2: essentially solid pour** — 284 regions, only 33 trace segments.
- **Inner 3: near-solid pour** — 283 regions plus 180 thin and 81 medium segments.
- **Top and bottom: all the routing.** Top has roughly 1794 thin, 706 medium and 525 wide segments; bottom has 1321, 1083 and 57.

In other words, **routing is entirely on the outer layers and all four internal layers are essentially uninterrupted plane copper**. For a board carrying an 80 MHz parallel bus past a 12-bit converter with a 10 W PA on the same substrate, that is exactly the right call — and it is the kind of decision that is hard to retrofit.

Placement follows the signal flow. Top side: the BNC and the two sense transformers at the top-left corner; the whole four-band low-pass bank with its hand-wound T37-6 toroids across the top third; Q1 and Q2 in the centre with large thermal pads and heatsink mounting holes, T3 and T4 immediately beside them; the three reed relays and the pi attenuator in a column on the right; the high-pass bank, the muxes and the MMIC across the middle-left; T5 and the ADC directly beside the FPGA module header; the power section top-right; audio, microphone and the three 3.5 mm jacks along the bottom edge.

Bottom side: the Pico, the **DAC**, the Darlington array, the GPIO expander, the LCD header, and the DAC/audio passives. Putting the DAC on the opposite face from the ADC with four plane layers between them is a deliberate isolation choice — the transmit path's switching noise has to get through the whole stack-up to reach the receive converter.

Rev 1.1 changelog, printed on the schematic: minor silk updates, improved Pico clearance, R59 changed to 2.4 W, wire lengths added, FT50-43 alternative for T3 and T4.

8.6 Stated performance

Everything quantitative in the repository is either design analysis (section 4.4) or a firmware calibration constant. **There is no measured test data** — no IMD, no sensitivity measurements, no spurious or harmonic sweeps, no efficiency figures, no current draw, no S-meter calibration table.

The claims that are stated:

Specification	Value	Source
Power output	5 W nominal, typically 6–10 W	README.md
Bands	80 m to 10 m	README.md
Receive coverage	0.5–30 MHz, reduced sensitivity below 3 MHz	README.md
Spectrum span	30 kHz (29.30 kHz derived)	README.md
Analogue gain ahead of the ADC	26 dB (20 dB MMIC + 6 dB transformer)	docs/README.md
Selectable attenuator	12 dB (–12.04 dB derived)	docs/README.md
DDC processing gain	31.1 dB, about 5.2 extra bits	docs/README.md
Effective in-channel resolution	about 17 bits	docs/README.md
System noise figure	8–10 dB	docs/README.md

MDS in 2.4 kHz	-131 to -134 dBm	docs/README.md
ADC clip, antenna-referred	-18 dBm = S9+55	docs/README.md
Modes	SSB, CW, FT8, digital via VOX; CESSB	README.md
AM receive	present in firmware (MODE_AM), not listed in README.md	dsp.h, uDST.ino
CESSB drive	$\times 1.5 = +3.52$ dB	derived from dsp.h
CW rise/fall	9.984 ms Gaussian	derived from cw.h
Notch depth/width	30 dB over ~ 26 Hz, -3 dB at ± 79 Hz	notch.h
Noise blanker latency	2.05 ms when enabled, 0 when off	dsp.h
Tuning resolution	0.01863 Hz	derived from the NCO

9 Appendix A: the numbers in one place

9.1 Rates and clocks

Quantity	Value
TCXO	24 MHz
PLL output, converter clock	80.000 MHz
ADC sample rate	80 MSPS, 12 bit two's complement
DAC sample rate	80 MSPS, 10 bit unsigned
CIC decimation	512
Rate between CIC and FIR	156.25 kHz
FIR decimation	5
DDC output rate	31 250 Hz , 16 bit signed I and Q
Total decimation	2560
DSP frame period	32 μ s = 8000 CPU cycles at 250 MHz
data_ready square wave	31.25 kHz, 16 μ s high / 16 μ s low
I/Q SPI clock	6 MHz (PIO), 5.5 μ s per 32-bit frame
Tuning-word SPI clock	8 MHz (hardware SPI1), ~4 μ s per write
Display SPI clock	20 MHz
I2C clock	400 kHz
Audio PWM carrier	3.90625 MHz
Audio / mic sample rate	31 250 Hz, 12 bit
FT8 audio rate	7 812.5 Hz ($\div 4$)
Panadapter FFT rate	30.5 blocks/s
Display refresh	20 Hz
NCO resolution	0.018626 Hz

9.2 Bit widths through the DDC

Point	Width
ADC	12, signed
Sign-extended for the mixer	16
Sine/cosine LUT	16, Q15

Mixer product	32
After >>>15 round	~13
CIC integrators and combs	48 (must match)
After >>>22 round, saturated	16
FIR coefficients	16, Q15
FIR accumulator	40
After >>>15, saturated	16

9.3 Bit widths through the DUC

Point	Width
From the Pico	16 field, low 10 bits used, signed
Zero-stuffed	10
TX FIR coefficients	16, Q15, ~5x DC gain
TX FIR accumulator	40
Saturated to the datapath	10
CIC comb width	16
CIC integrator width	40
After SCALE_SHIFT = 18	10
Mixer product	32
After >>>15, offset, saturated	10, unsigned

9.4 Gain accounting, receive

Stage	Gain
Quadrature mix of a real signal	×0.5
CIC $512^3 = 2^{27}$ with >>>22	×32
FIR, Q15 coefficients summing to 32768, >>>15	×1
ADC codes to I/Q codes	×16

Full-scale ADC (± 2047) becomes $\pm 32\,752$ — 16-bit full scale within 0.004 dB.

9.5 Firmware filter inventory

All designed at $f_s = 31\,250$ Hz, all linear-phase FIR, all 60 dB stopband unless noted, all generated with arc.id.au/FilterDesign.html.

Function	Spec	Taps	Real MACs	Used by
----------	------	------	-----------	---------

lpf_2200f_rx	LPF 2200 Hz	255	255	bandwidth 1 (labelled 2000 Hz)
lpf_2400f_rx	LPF 2400 Hz	255	255	bandwidth 2 (2200 Hz)
lpf_2600f_rx	LPF 2600 Hz	255	255	bandwidth 3, default (2400 Hz)
lpf_2800f_rx	LPF 2800 Hz	255	255	bandwidth 4 (2600 Hz)
lpf_3000f_rx	LPF 3000 Hz	255	253	bandwidth 5, digital RX, FT8 RX
lpf_2600f_tx	LPF 2600 Hz	255	255	microphone, pre-Hilbert
lpf_3000f_tx	LPF 3000 Hz	255	253	digital TX
lpf_2600if_tx	LPF 2600 Hz	125	125	CESSB , I re-filter
lpf_2600qf_tx	LPF 2600 Hz	125	125	CESSB , Q re-filter
bpf_700f	BPF 600–800 Hz	255	255	CW RX and both CW decoders
hpf_200f	HPF ~200 Hz	255	255	SSB RX high-pass
lpf_fs8f_rx1	LPF 3906 Hz ($f_s/8$)	127	97	AM , I channel
lpf_fs8f_rx2	LPF 3906 Hz ($f_s/8$)	127	97	AM , Q channel
lpf_3000f	LPF 3000 Hz	127	127	AM , post-detector audio

Non-FIR blocks:

Function	Structure	Coefficients
dcf, dc1f, dc2f	single-pole IIR high-pass, three instances	$k = 0.05$, ~200 Hz corner
fap1f	3 x 1st-order all-pass, +45°	$k = 0.98325, 0.88497, 0.59331$ (84 / 607 / 2539 Hz)
fap2f	3 x 1st-order all-pass, −45°	$k = 0.07102, 0.78470, 0.94391$ (8628 / 1200 / 287 Hz)
jnr_maf1..4	8-tap boxcar, float	the noise-reduction cascade
ma1, ma2	16-tap boxcar, int32	SWR forward and reflected smoothing
notch (in notch.h)	two cascaded 2nd-order IIR notches	designed at runtime from f_0 , bw = 100 Hz, depth = 30 dB

Thirteen further FIR and moving-average functions are defined but referenced nowhere — lpf_fs8_1/2, lpf_fs4f_rx1/2, hpf_fs4f_rx1/2, hpf_fs8f_rx1/2, hpf_300f, bpf_45p, bpf_45n, ma4i, ma4q. They are legacy from the MBPTRX (Tayloe-architecture) predecessor. Being `static`, the linker drops them.

10 Appendix B: pin and port maps

10.1 Pico GPIO

GPIO	Macro	Function
0	PIN_PTT	Microphone PTT, active low, pull-up
1	PIN_ENCBUT	Encoder pushbutton, pull-up
2, 3	PIN_ENCA, PIN_ENCB	Rotary encoder quadrature
4, 5	PIN_SDA, PIN_SCL	I2C at 400 kHz
6, 7, 8, 9	PIN_IQ_SCK, PIN_IQ_MOSI, PIN_IQ_MISO, PIN_IQ_CS	PIO I/Q bus to the FPGA
10, 11, 13	PIN_TW_SCK, PIN_TW_MOSI, PIN_TW_CS	Hardware SPI1, tuning word
12	PIN_DR	FPGA data_ready, the 31.25 kHz heartbeat
14, 15	PIN_AUPWML, PIN_AUPWMH	Audio PWM, low and high 6 bits
16	PIN_MUTE	Audio mute, low = muted
17–21	—	LCD CS, SCK, MOSI, DC, RST on SPI0
22	PIN_TXEN	TX enable: PA bias LDO, FPGA tx_rx, T/R relay
23	PIN_REG	Pico SMPS mode select (high = forced PWM)
26, 27	PIN_PADA, PIN_PADB	CW paddle dit / dah, pull-ups
28	PIN_MIC	Microphone into ADC channel 2

10.2 FPGA pin assignments

Signal	Pins
adc_in[11:0]	25, 26, 27, 28, 29, 30, 33, 34, 35, 36, 37, 38 (LSB at 38)
clk80	39
spi_tw_sck / mosi / cs_n	40 / 41 / 42
dac_out[9:0]	77 down to 68
tx_rx	57
spi_iq_sck / mosi / miso / cs_n	56 / 55 / 54 / 53
data_ready	51
sys_clk	52 (pull-up)

leds_n[1:6]	16, 15, 14, 13, 11, 10 — LVC MOS18 , not wired on this board
-------------	---

10.3 TCA9534 port assignments

Bit	Net	Drives
0	LPF1	80/60 m low-pass, via ULQ2003
1	LPF2	40/30 m low-pass, via ULQ2003
2	LPF3	20/17 m low-pass, via ULQ2003
3	LPF4	15/12/10 m low-pass, via ULQ2003
4	ATTEN	Both attenuator relays, via ULQ2003
5	HPF1	Mux select S0, direct
6	HPF2	Mux select S1, direct
7	—	unused

10.4 Bands

Each band keeps its own frequency in RAM, seeded from the table below. The power-on band is 40 m, so 7.100 MHz is where the radio starts. None of these are saved to EEPROM.

Band	Range (MHz)	Seed frequency
80 m	3.5 – 3.8	3.600
40 m	7.0 – 7.3	7.100 (power-on band)
30 m	10.1 – 10.15	10.120
20 m	14.0 – 14.35	14.200
17 m	18.068 – 18.168	18.100
15 m	21.0 – 21.45	21.200
12 m	24.89 – 24.99	24.900
10 m	28.0 – 29.7	28.500
SWL	0.5 – 30	5.000, receive only

11 Appendix C: discrepancies and gaps found in the source

These are worth knowing before you modify anything. None of them is a functional bug in normal use, but several would mislead a reader of the source, and two are behavioural surprises.

11.1 Comment/code mismatches (since fixed)

The mismatches below were present at the time this document was compiled; the stale comments have since been corrected in the source to match the code. The table is kept as a historical record.

Where	Comment says	Reality
<code>uDST.ino:832 (setup)</code>	“run DSP on core 1”	DSP <i>is</i> on core 1 — but the comment is in <code>setup()</code> , which is core 0
<code>uDST.ino:1059 (setup1)</code>	“run UI on core 1”	Wrong. Core 1 is DSP, core 0 is UI. Authoritative markers are <code>loop1():2262</code> and <code>loop():4796</code> , plus the history entry “1.0.250 move DSP to core 1”
<code>stackpaint.h:16</code>	“PAINT_SIZE is 7 KB of the 8 KB block”	<code>STACKPAINT_SIZE</code> is actually 3 KB
<code>filter.h, hpf_200f</code>	“Lo: 300”	Coefficients differ from <code>hpf_300f</code> ; it really is the 200 Hz filter. Comment copied
<code>notch.h:25</code>	“RP2350 @ 240 MHz ... 7680 cycle frame”	The build is 250 MHz, so the frame is 8000 cycles
<code>ft8.h:118</code>	“2x zero-padded”	Nothing is zero-padded. The frame is twice the symbol length
<code>DUC18.v</code> header	references <code>iCE40</code> <code>SB_MAC16</code> and <code>synth_ice40 -dsp</code>	Legacy from the prototype. The target is Gowin. <code>DDC26.v</code> 's header was corrected; <code>DUC18.v</code> 's was not
<code>radio_top.v:33, :54</code>	“20 MHz crystal/TCXO”, “20 MHz -> rPLL -> 80 MHz”	The TCXO is 24 MHz. Same legacy as the <code>p11_20_80.v</code> filename
<code>radio_top.v:28</code>	lists “ <code>DDC20.v</code> ” as a project source	The file is <code>DDC26.v</code>
<code>verilog/uDST81/DDC23.v:2</code>	headed “ <code>DDC22.v</code> ”	The file is <code>DDC23.v</code>
<code>uDST.ino:541</code>	“~31.25 kHz (80 MHz) or ~31.64 kHz	The 81 MHz build also produces exactly 31 250 Hz (432 x 6 = 2592). The 31.64

	(81 MHz)”	kHz figure is wrong
README.md:36	NCO tuning has “sub-millihertz resolution”	$80\text{ MHz}/2^{32} = 0.0186\text{ Hz}$ — three orders of magnitude coarser than the claim, though still far finer than anything audible
docs/README.md:7 VS DDC26.v:15	full-scale I/Q is $\pm 32\,752 / \pm 32\,734$	A 0.005 dB disagreement from Q15 rounding. DDC26.v marks its figure “confirmed in simulation”, so trust 32 734

11.2 Behavioural surprises

The CW sidetone pitch is hard-coded at 700 Hz. `CW::sidetone()` contains `static const uint64_t tone = 700ull`; and its level is likewise a hard-coded 3. Meanwhile `radio.sidetone` is a menu setting (500–850 Hz), is validated, is stored in EEPROM, and *is* used — but only to offset the **receive** NCO in `set_frequency()`. So if you set the sidetone to anything other than 700 Hz, the pitch you hear on receive and the pitch you hear while transmitting will differ.

Filter state is shared between receive and transmit. `fap1f`, `fap2f`, `dcf`, `dc1f` and `dc2f` hold static state and are called from both the receive demodulators and `process_mic/process_dig`. This is safe, because the two paths are mutually exclusive, but a mode or T/R transition inherits stale filter state. That is precisely why there are `mute()` and `delay()` sequences around every transition.

The I/Q halves swap meaning between receive and transmit. On receive the 32-bit frame is `{I, Q}`; on transmit the firmware writes its `tx_q` into the half the FPGA reads as `tx_i`. See section 4.6.2. Not a bug — it is part of the sideband selection — but it will catch anyone tracing a signal.

`tx_safe` is not a permanent latch. The boot interlock (transmit disabled unless PTT and both paddles read high at power-on) is cleared unconditionally by switching to either digital mode, which sets `radio.tx_safe = true`.

User_Setup.h defines `TFT_MISO` as GP16, which is also `PIN_MUTE`. The LCD is wired and driven as four-wire SPI (CS, SCK, MOSI, DC, RST on GP17–21) and MISO is left unconnected on the board, so nothing breaks — but the TFT_eSPI configuration nominally claims a pin the firmware uses for audio mute. Worth knowing before enabling any display read operation.

11.3 Documentation gaps

Not stated anywhere in the repository:

- DC input voltage range
- Current draw (quiescent, receive, or transmit)
- TCXO jitter or phase-noise specification
- FPGA LUT and BRAM utilisation
- Exact design cutoffs and ripple for the low-pass and high-pass filter banks
- PCB stack-up (dielectric thicknesses, copper weight, impedance targets)

- Any measured RF performance data at all

11.4 BOM inconsistencies

Item	Issue
U5, U6	Name column says SN74CBT3253CDR (SOIC-16, matching the footprint); manufacturer-part column says SN74CBT3253PWR (TSSOP-16). One is wrong
K9–K11	Name and schematic say HE721C1210; BOM manufacturer part says HE721C1200 — a coil-voltage variant mismatch
U16	ULQ2003D1013TRY VS ULQ2003D1013TR
L1–L6, L13–L16, L21, L22, L27, L28	All carry manufacturer part AISM-1812-1R0K (a 1.0 μ H 1812 part) despite values of 330 nH, 680 nH, 1.2 μ H and 2.2 μ H and a stated 1206/0806 footprint requirement. These are placeholders — only L20 and L26 (10 μ H) have a real part number
Most R and C	Part-number column blank or “?”. The BOM specifies values; BOM/README.md specifies the class (1206 thick film, NP0/C0G, at least 100 V for the low-pass filters)
D1 orientation	Could not be resolved from the schematic render. Verify on the board
Relay contact assignment	NO vs NC for HE721 and ALQ112 is inferred from topology, not read from the symbols. The K9–K11 pin numbers other than “pin 14 to ground” are read from a zoomed render, not from legible text
Mic output resistor	The 2 k resistor forming the 8 kHz output pole is either R45 or R48 — both are 2 k and the render is ambiguous
Datasheet-only facts	The AD9236’s MODE-to-format mapping and 2 V p-p span, the AD9740’s 1.2 V internal reference and speed grade, the OPA2674’s current limit, the FQD11P06’s and MC7805’s voltage ratings, T5’s turns ratio (the repo states only “6 dB”), and the ST7789 panel’s 1.14 inch active area are all standard part behaviour, not repository content
PCB placement	The placement description in section 8.5 is read off the board renders. The layer counts, dimensions, and layer-usage figures are measured directly from the Gerbers

11.5 Architectural notes for a reviewer

- No memory barriers** on any cross-core volatile handshake. Correctness rests on volatile plus generous dead zones. Fine at these rates on coherent SRAM, but it is an assumption, not a guarantee.
- The FT8 decimate-by-4 has no dedicated anti-alias filter** beyond 1pf_3000f_rx. The new Nyquist is 3906.25 Hz against a 3000 Hz corner — adequate, with 906 Hz of margin.
- FPGA_CLK is hard-coded to 80 MHz**, and switching to the alternative FPGA build is easier than a firmware comment suggests. The repository ships a second build

(`verilog/uDST81/`, 27 MHz to 81 MHz PLL, `DDC23.v`, a 252-tap FIR) which decimates by **$432 \times 6 = 2592$** — so 81 MHz also gives exactly **31 250 Hz**. Only `FPGA_CLK` (the tuning-word denominator) would need to change; `SAMPLERATE` and every filter coefficient set stay as they are. A comment at `uDST.ino:541` once claimed 31.64 kHz, which made this look harder than it is; that comment has since been corrected. The 81 MHz variant is marked “untested in production”.

4. **CESSB is a single-iteration approximation** of the published two-iteration method. See section 5.2.
5. **FT8 transmit is unshaped MFSK**, not GFSK. See section 7.8.
6. **FT8 has no subtractive or multi-pass decoding**. Expect fewer decodes than WSJT-X in crowded conditions.
7. **Noise reduction is a low-pass, not a noise estimator**. The name is more ambitious than the algorithm. See section 7.4.

12 Appendix D: where to look in the source

12.1 Repository layout

BOM/	BOM_uDST_2026-09-08.md and .txt, plus build notes in README.md
docs/	Schematic PDF (Rev 1.1), block diagrams, and the gain/noise analysis in
README.md	
PCB/	Gerber archives (main board plus front and back panels), board renders
src/	Arduino sketch and all headers, plus the bundled TFT_eSPI2 library
verilog/	The 80 MHz build: radio_top.v, DDC26.v, DUC18.v, pll_20_80.v, coefficient
hex, bitstream	
verilog/uDST81/	The untested 81 MHz variant

12.2 Verilog

File	Contents
radio_top.v	Clocking, reset, converter I/O, both SPI slaves, the data_ready generator, status LEDs
DDC26.v	nco_sincos_rx, mac16_reg, fir255_dec5_rx, ddc_top (mixer, CIC, scaling)
DUC18.v	upsampler_zero_stuff5_tx, fir_serial_tx, cic_interp_3rd_tx, nco_iq_mix_lut_tx, duc_top
pll_20_80.v	The single rPLL instance, 24 MHz to 80 MHz
radio_top.cst	Pin assignments, I/O standards, drive strengths, pull modes
sin_lut.hex	1024-entry Q15 sine table (loaded four times: two NCOs x two ports)
fir_rx255_coeffs.hex	256 words — 255 taps plus a mandatory trailing zero

12.3 Firmware, by size

File	Lines	Contents
filter.h	5934	Every FIR (unrolled, coefficients as literals), DC blockers, Hilbert all-passes, boxcars
uDST.ino	5331	Both loop()s, pin map, display, menu handling, metering, band switching, FT8 sequencing
fft2500.h	4709	The 2500-point real FFT and its twiddle tables, plus the Hann window
spectrum.h	4595	The 1024-point integer FFT, window, DC removal, magnitude, log table, colour map
ft8.h	1678	Protocol constants, waterfall, sync search, LDPC, CRC, message packing, encode
CW.h	1437	Gaussian envelope table, sidetone DDS, keyer envelope state machine
util.h	884	SAMPLERATE, compile-time colour bit-reversal, small lookup tables

<code>cwdecode1.h</code>	567	Adaptive-threshold CW decoder
<code>menu.h</code>	516	Menu structure and option tables
<code>dsp.h</code>	484	AGC, S-meter, noise blanker, all demodulators, mic processor, CESSB
<code>cwdecode2.h</code>	430	Schmitt-trigger CW decoder
<code>notch.h</code>	173	The twin-zero notch
<code>TCA9534A.h</code>	161	GPIO expander driver
<code>ft8ui.h</code>	157	FT8 timing constants and UI state enums
<code>asyncSPI.h</code>	155	The PIO I/Q SPI master
<code>stackpaint.h</code>	89	Stack high-water measurement
<code>tuningWordSPI.h</code>	63	The hardware-SPI tuning-word writer
<code>mcp3021.h</code>	58	I2C ADC driver
<code>Rotary.cpp/.h</code>	144	Ben Buxton state-table encoder decoder

12.4 Build configuration

Firmware: Arduino IDE with the Earle Philhower `arduino-pico` core. Board: Pi Pico 2. CPU speed: 250 MHz. Optimise: `-O3`. USB stack: **No USB**. Flash size: 4 MB (no filesystem). The bundled `TFT_eSPI2.zip` must be installed as a library, and `User_Setup.h` configures it for the ST7789.

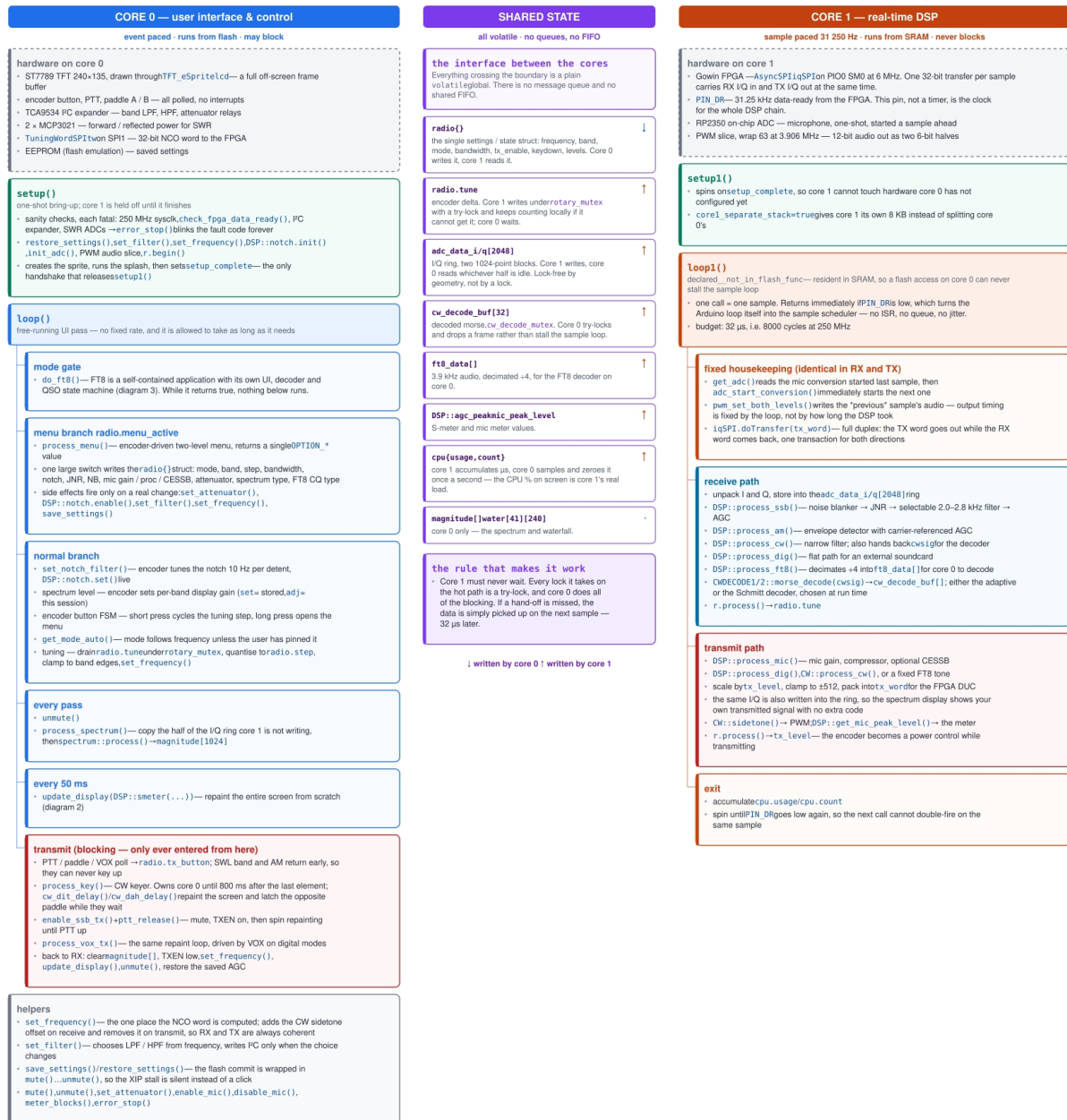
FPGA: Gowin EDA, device `GW1NR-LV9QN88PC6/I5`. Under Place and Route, Dual-Purpose Pin, check “Use SSPI as regular IO”. Program with `openFPGALoader -b tangnano9k -f uDST.fs`.

12.5 Program structure diagram

uDST — program structure

uDST_12.ino (V1.5.250) — RP2350 @ 250 MHz, dual core. Core 0 runs the user interface, core 1 runs the DSP.

Diagram 1 of 3 - see also: display painter, FT8 sub-application



Ian Mitchell VK7IAN - uDST direct-sampling transceiver - generated from uDST_12.ino

The core structure, from docs/uDST_structure_1_cores.png. SVG versions of all three structure diagrams are in docs/ at higher resolution.

13 Closing note

There is a coherent design philosophy running through this radio, and it is worth naming.

Every decision that could be moved out of the analogue domain has been. There is one active device between the antenna and the converter. There are two adjustable components on the whole board, both of them PA gate bias trimmers. Sideband selection is an argument order. Bandwidth is a function pointer. The FPGA's entire configurable state is a single 32-bit integer. The scheduler is a wire.

The consequence is that the radio's performance lives in text files. The 255-tap FIR does not age. The quadrature does not drift. Unit two behaves exactly like unit one, with no alignment procedure, because there is nothing to align.

The second thing worth naming is that the numbers are not accidental. The CIC output shift is 22 because that puts digital full scale within 0.004 dB of the converter's own, and because the five bits it keeps are exactly the five bits that decimation earned. The analogue gain is 26 dB because that lands the antenna-referred clipping point at S_9+55 , which is the same point the digital scaling chose. The noise-blanker's 32-sample run limit is 1 ms because impulses are shorter than that and voices are longer. The FT8 auto-calibration steps by 4 s because the sync search covers 4.72 s. Read enough of the source and a pattern emerges: almost every constant has a reason, and most of those reasons are written down next to it.

That, more than the architecture, is what makes the uDST worth studying.